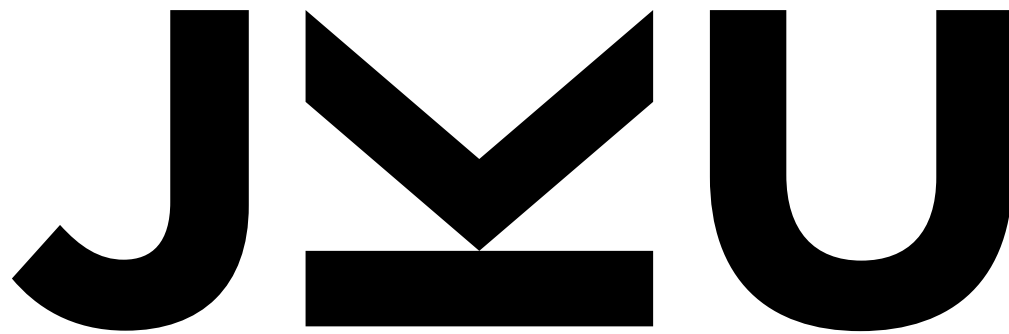




**JOHANNES KEPLER
UNIVERSITY LINZ**

Advanced CNN architectures



Günter Klambauer
LIT AI Lab & Institute for Machine Learning

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Strasse 69
4040 Linz, Austria
jku.at

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

11. Advanced CNN architectures

- 11.1. Image Classification and Image Recognition
- 11.2. Semantic Segmentation
- 11.3. Object Detection and Localization
- 11.4. Instance Segmentation (brief!)

11.1. Image Classification and Image Recognition

- 11.1.1 AlexNet (2012)
- 11.1.2 ZFNet (2013)
- 11.1.3 VGGNet (2014)
- 11.1.4 Network in Network (2014)
- 11.1.5 GoogLeNet (2015)
- 11.1.7 Highway Networks (2015)
- 11.1.8 ResNet (2015)
- 11.1.9 Pre-activation ResNet (2016)
- 11.1.10 ResNeXt (2016)
- 11.1.11 Xception (2016)
- 11.1.12 DenseNet (2016)
- 11.1.13 Squeeze and Excitation Net (2017)
- 11.1.14 MobileNet V2 (2018)
- 11.1.15 EfficientNet (2019)
- 11.1.16 The vision transformer (2020)
- 11.1.17 ConvNeXt (2022)
- 11.1.18 New learning paradigms through self-supervised learning (2020)
- 11.1.19 Conclusions

Important deep networks for computer vision

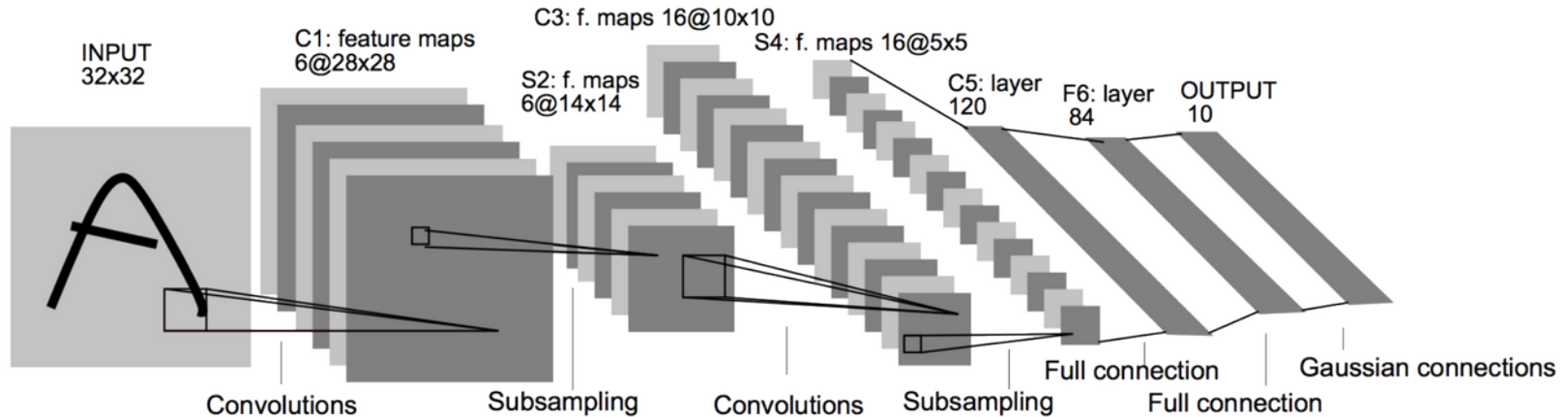
- AlexNet
- VGG
- Inception networks and GoogLeNet
- ResNet
- ResNext
- Xception
- DenseNet
- MobileNetV1 and V2
- EfficientNet
- RegNet
- Vision transformers
- ConvMixer
- ConvNext

Starting point: LeNet-5

- Note how convolutions work with multiple feature maps (channels):

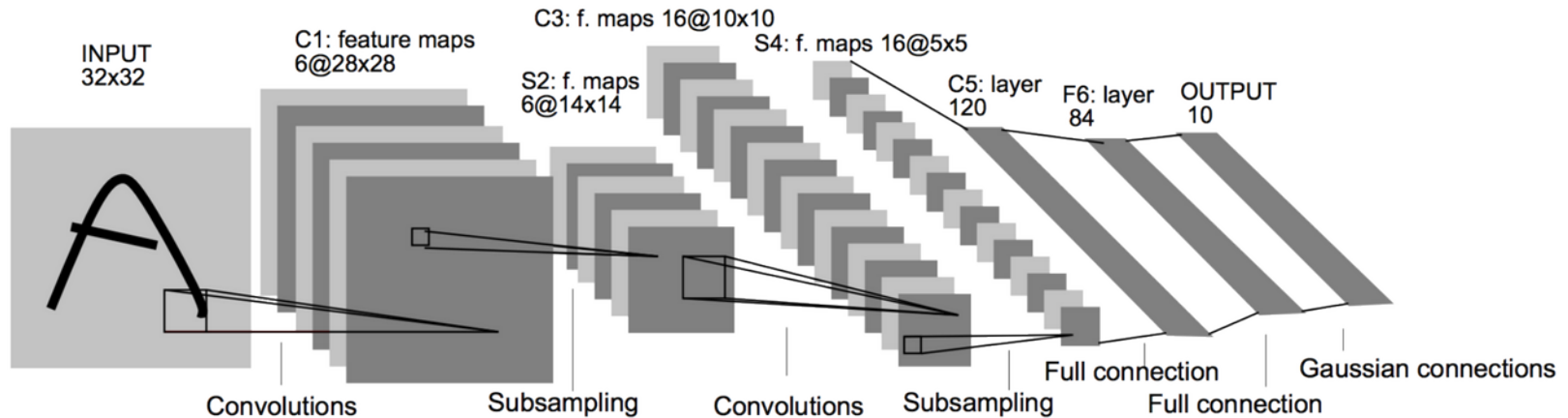
$$s_{a,b} = \sum_{c=0}^{C-1} \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j,c} x_{a+i,b+j,c} = (\mathbf{W} * \mathbf{x})_{a,b}$$

Example: LeNet



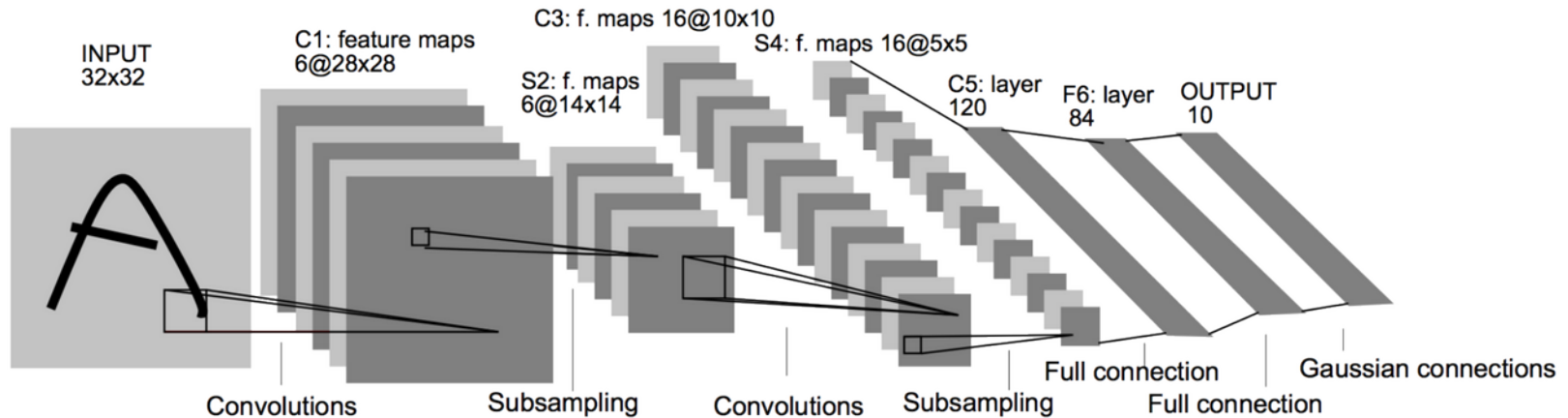
- **Input:** gray scale image of size 32x32 or 32x32x1
- **C1:** ConvLayer with 6 kernels of size (5,5) and stride 1
 - Output dimensions are: 28x28x6
 - Number of trainable parameters: $(1*5*5+1)*6=156$
- **S2:** Pooling with size (2,2) and stride 2.
 - Output dimensions are: 14x14x6

Example: LeNet



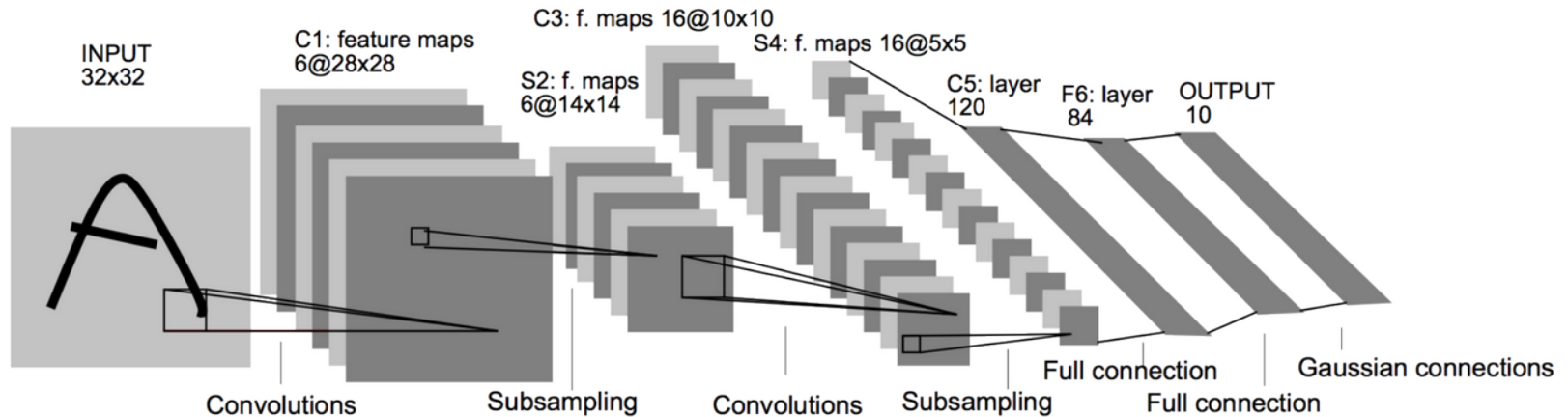
- **C3:** ConvLayer with 16 kernels of size (5,5) and stride 1
 - Output dimensions are: 10x10x16
 - Number of trainable parameters: $(6*5*5+1)*16=2416$
- **S4:** Pooling with size (2,2) and stride 2.
 - Output dimensions are: 5x5x16

Example: LeNet



- **C5:** ConvLayer with 120 kernels of size (5,5) and stride 1
 - Output dimensions are: 1x1x120
 - Practically means “full-connection”
 - Number of trainable parameters: $(16*5*5+1)*120=48120$
- **F6:** Fully-connected layer with 84 units.
 - Number of trainable parameters: $(120+1)*84$

Example: LeNet



- **Output:** Fully-connected layer with 10 units.
 - Number of trainable parameters: $(84+1)*10$

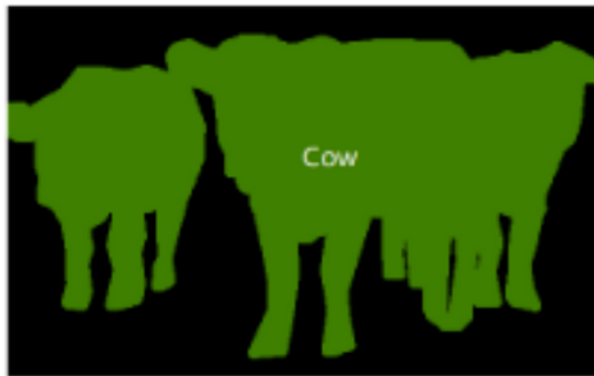
Fundamental computer vision tasks



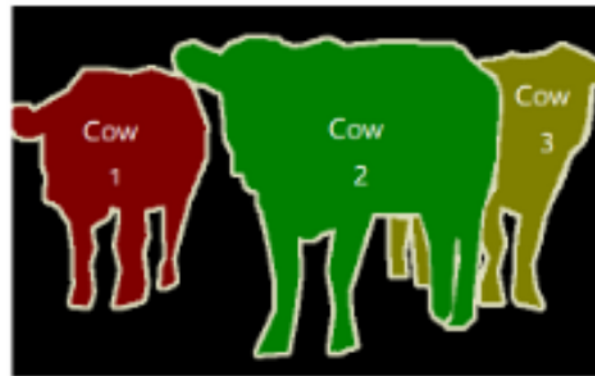
(a) Image Classification



(b) Object Detection



(c) Semantic Segmentation



(d) Instance Segmentation

ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)

- 14 million images hand-annotated
 - 20,000 categories
- 1 million images with bounding boxes
- Annual challenge since 2010
- Challenge
 - uses 1,000 non-overlapping categories
 - ~1.2 million images
- Top-1 and Top-5 accuracy evaluated

ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)

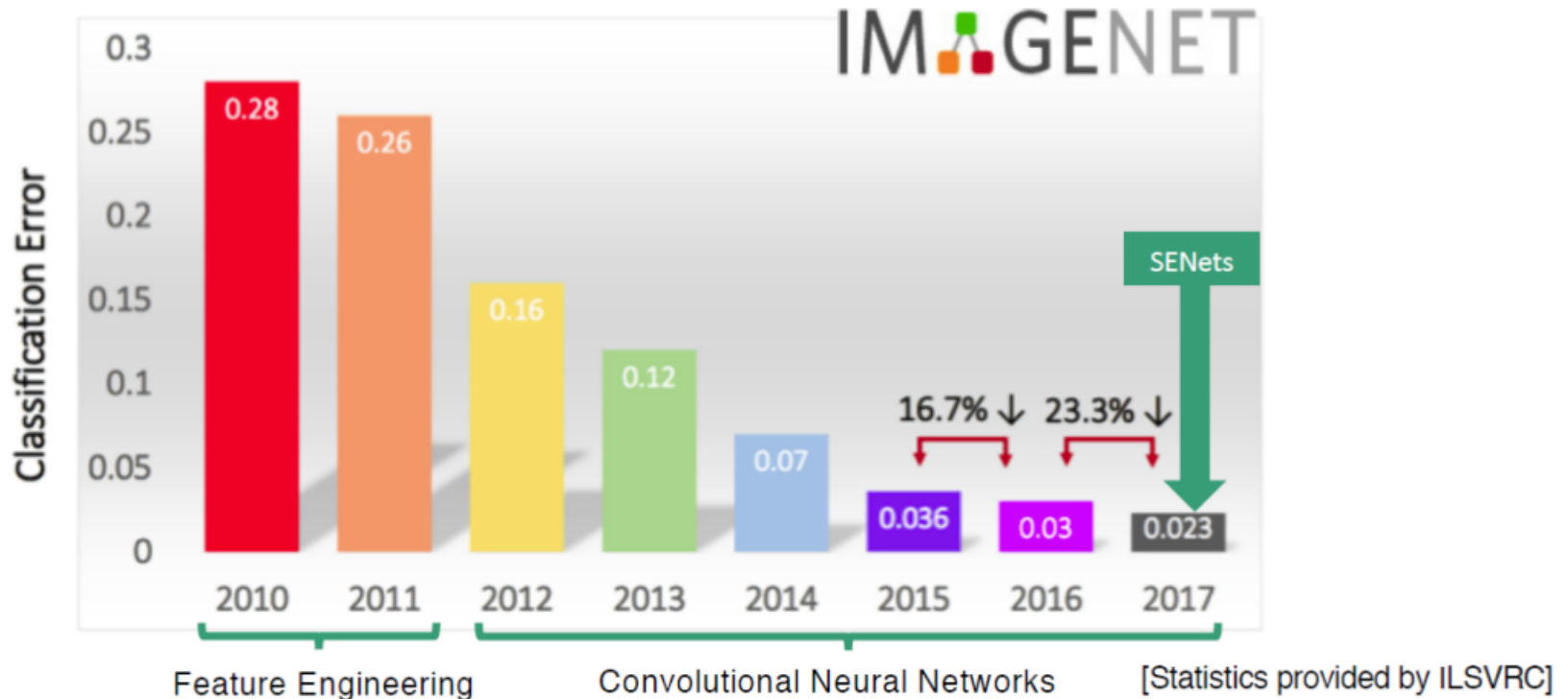
- 14 million images hand-annotated
 - 20,000 categories
- 1 million images with bounding boxes
- Annual challenge since 2010
- Challenge
 - uses 1,000 non-overlapping categories
 - ~1.2 million images
- Top-1 and Top-5 accuracy evaluated



ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)



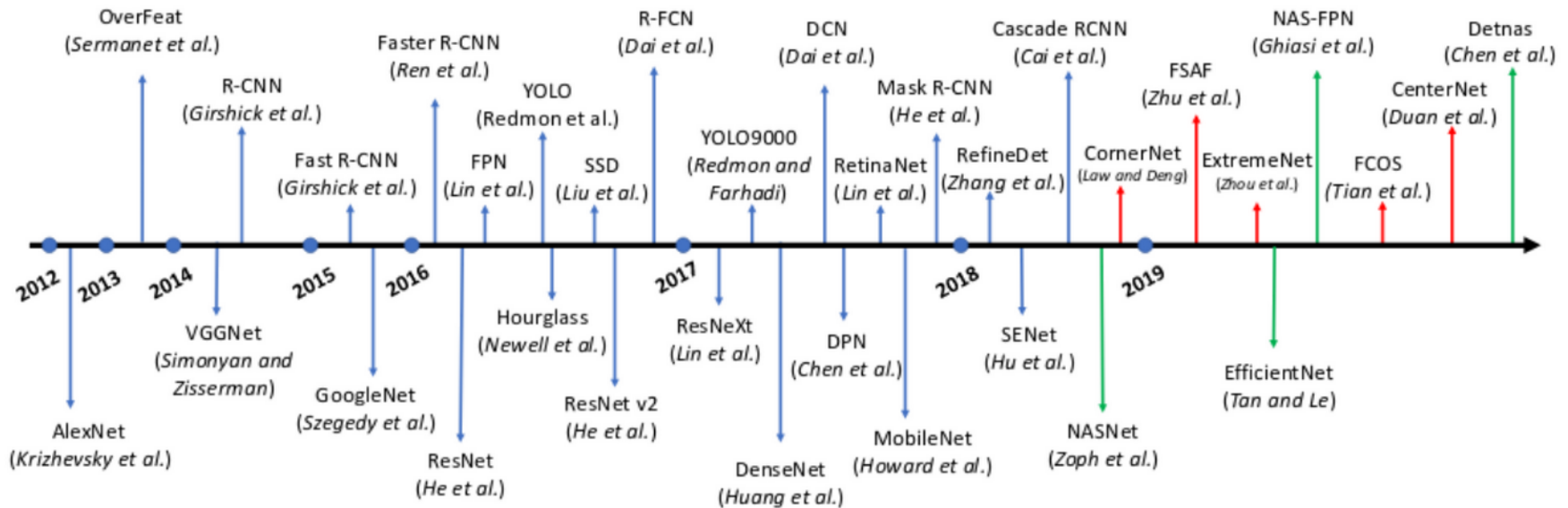
ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)



Progress in computer vision translates to other fields

- A large, relevant data set was publicly available
- Enables extraction of semantic information from images
- The key is to learn meaningful features
- Improved performance on ImageNet translates to other classification data sets
- Pre-trained networks on ImageNet are often backbones or feature extractors for other tasks
- The use of pre-trained models speeds-up training considerably

Milestones in CNN architectures and Object Detection methods

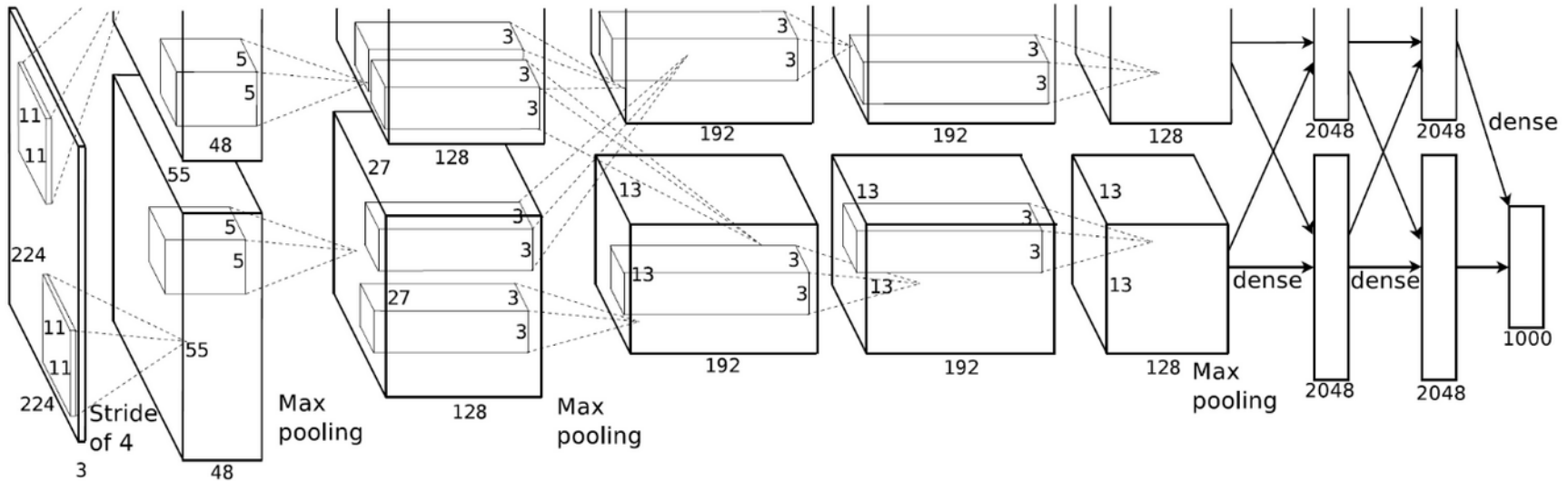


AlexNet (2012)

- AlexNet (Krizhevsky et al., 2012) won the ILSVRC-2012 competition by a large margin
 - 15.3% error rate vs 26.2% by second-best method
- Principle design of LeNet, but for 224x224 images
- GPU training
- Other algorithmic advances

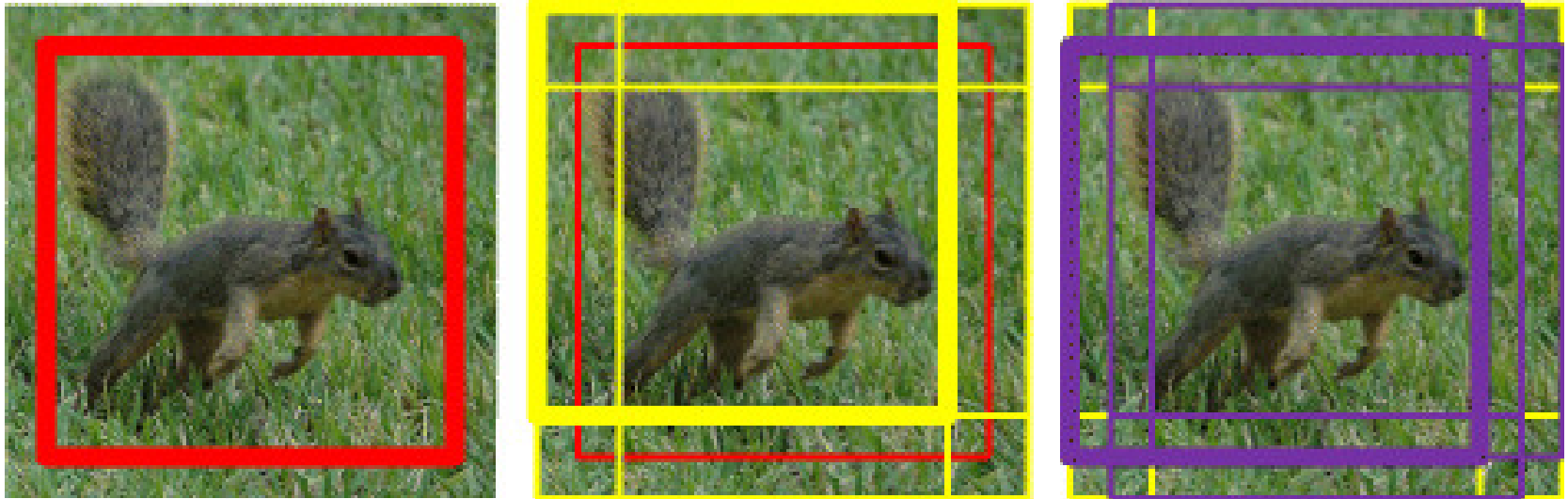
Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In Advances in neural information processing systems (pp. 1097-1105).

AlexNet (2012)



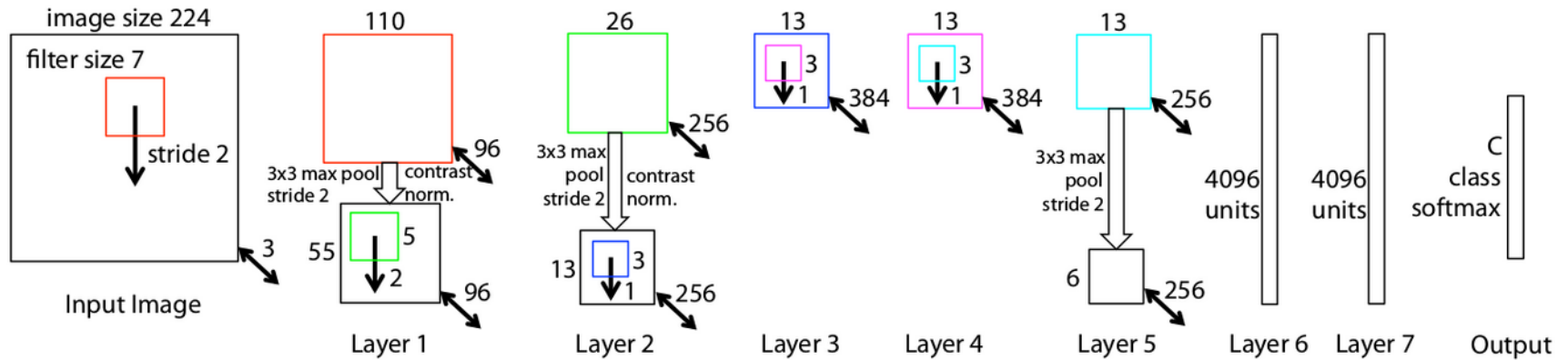
- The AlexNet architecture and training
 - ReLU activations
 - Dropout in fully-connected layers
 - Overlapping max-pooling layers 3x3 with stride 2
 - Multi-GPU training
 - Data Augmentation

Cropping as regularization and data augmentation technique



- Cropping: Selecting a sub-image each time the image is passed through the network
 - Data augmentation technique
- Test time: average over multiple crops

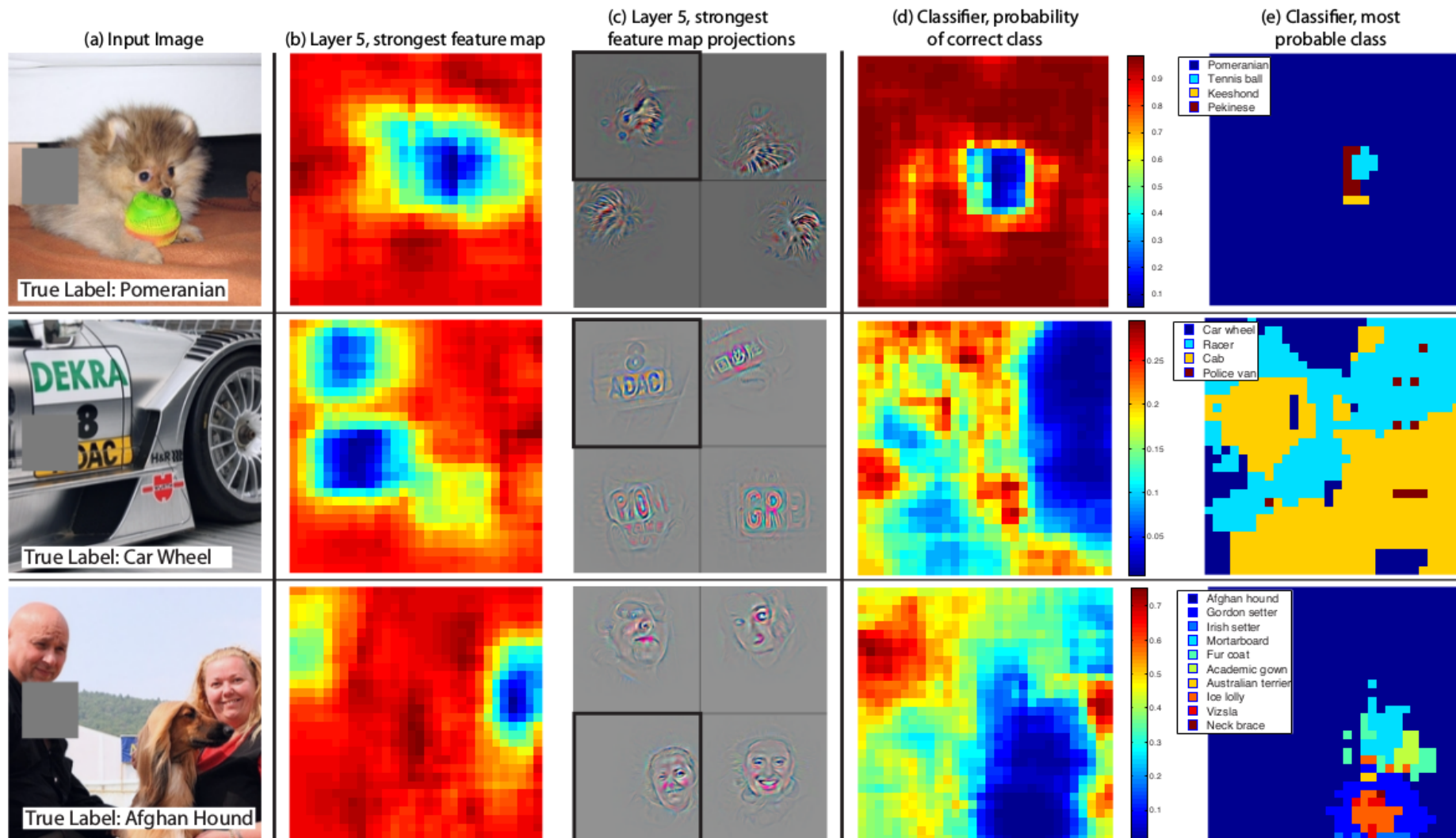
ZfNet (2013)



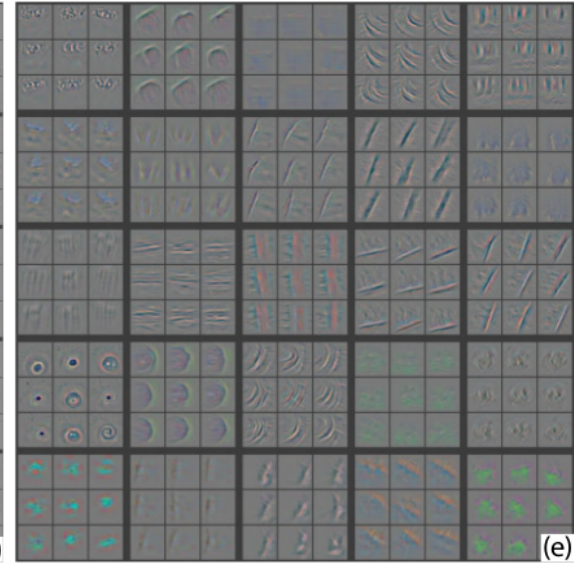
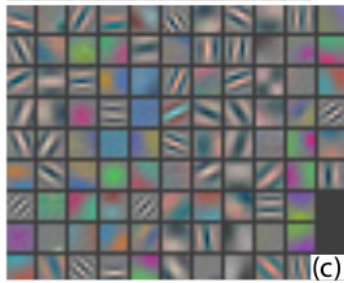
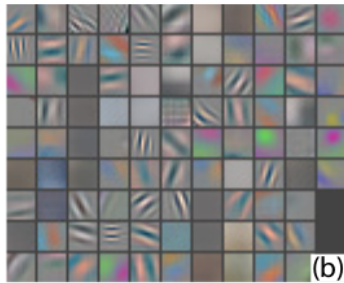
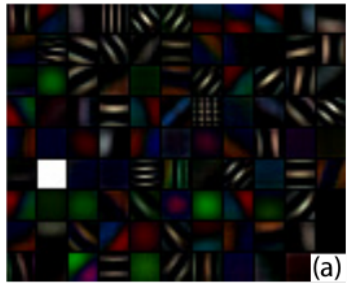
- Considered the winner of ILSVRC-2013; error rate 13.5%
- Visualization technique developed to get insights into AlexNet
- Specific changes to architecture by these insights
 - 7x7 kernel in first layer
 - Stride 2 in first two layers
- First attempts at transfer learning to other imaging datasets (CalTech-101).
 - Results: features learned by ZfNet can be transferred

Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Springer, Cham.

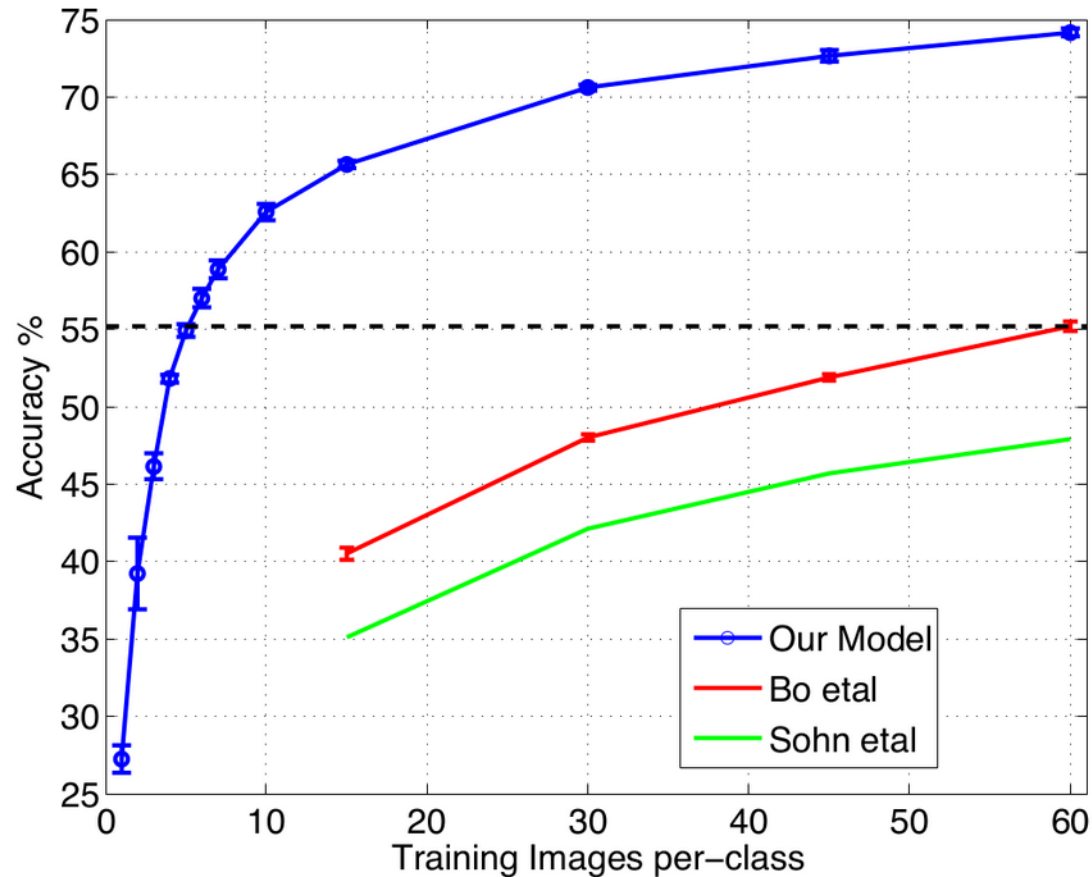
ZfNet (2013)



ZfNet (2013)



ZfNet (2013): Transfer learning on Caltech-256

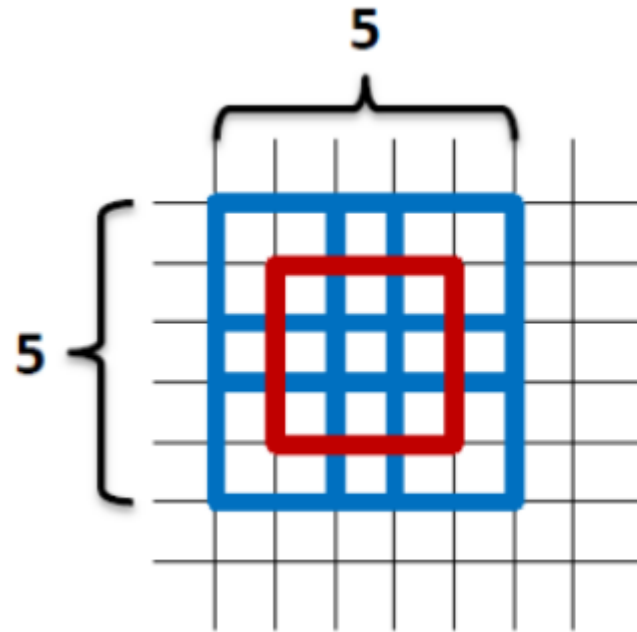


VGGNet (2014)

- VGGNET (Simonyan & Zisserman, 2014) achieved second place in ILSVRC-2015
- Submission by Visual Geometry Group (VGG) of Oxford
- Only 3x3 convolutions are used (stride 1)
 - Network had to become deeper
- Baseline VGG-11 (8 conv, 3 fc layers)
- Architecture and hyperparameter search
 - Tests with 1x1 convolutions (reduction of channels)

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

VGGNet (2014)



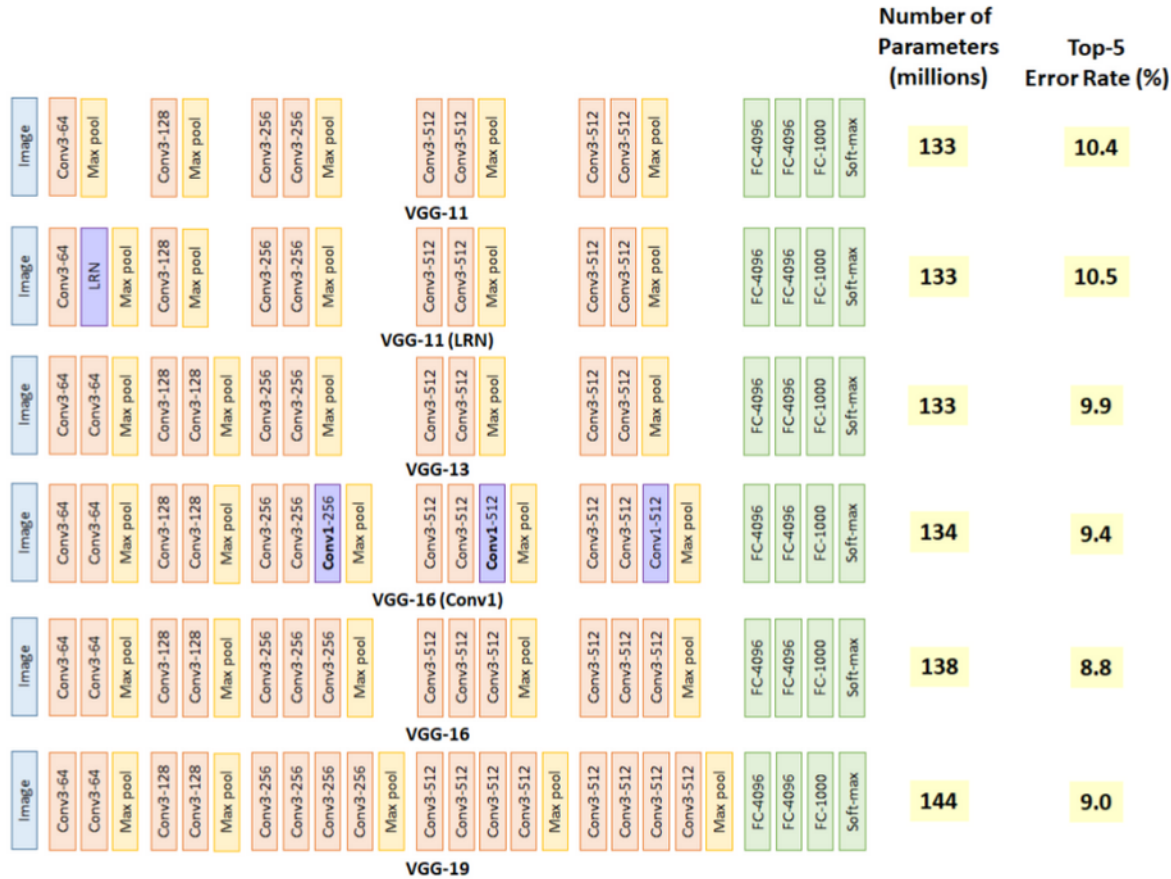
1st 3x3 conv. layer



2nd 3x3 conv. layer

- Replacement of 5x5 kernel with two 3x3 layers
- Receptive field is the same

VGGNet (2014)



Network in Network (2014)

- Network In Network (Lin et al., 2014) did not participate in ILSVRC
- Good results on CIFAR-10 and CIFAR-100
- Removed FC layers at end by *global average pooling*
 - Global average pooling:

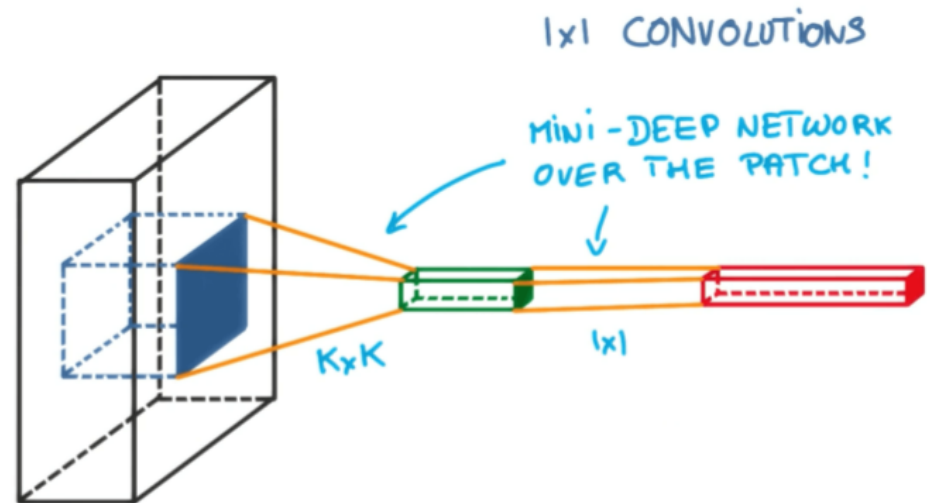
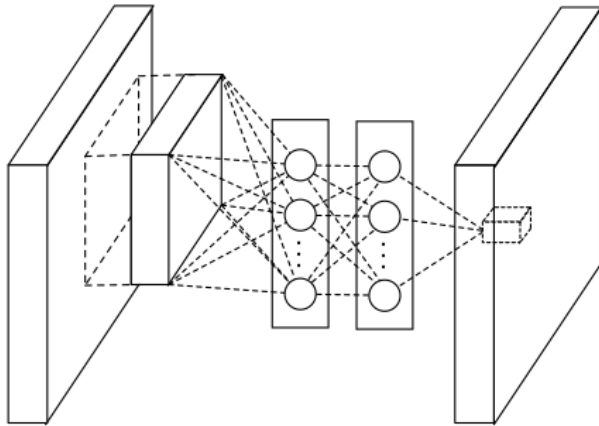
$$\mathbf{s}_c = \text{gap}(\mathbf{x})_c = \frac{1}{AB} \sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \mathbf{x}_{a,b,c}$$

$\mathbf{x} \in \mathbb{R}^{A \times B \times C}$, width: A , height: B , feature maps: C

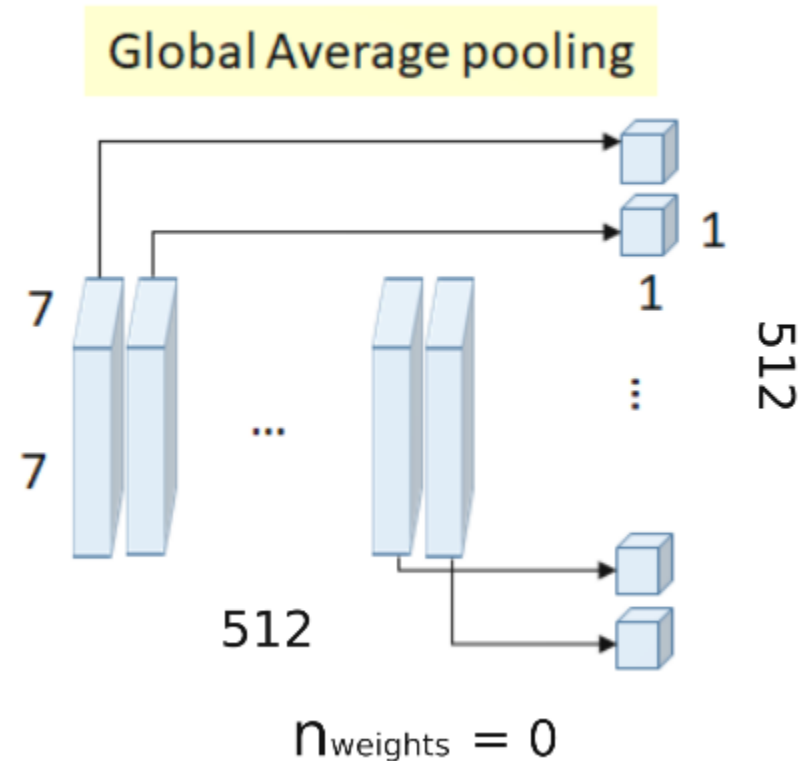
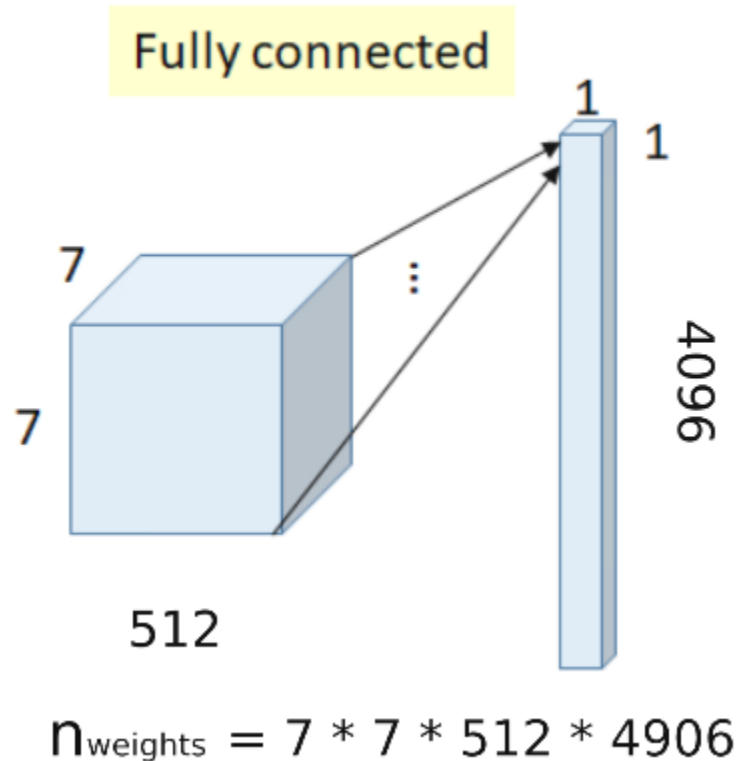
- 1x1 convolutions as “mini multi-layer perceptron”.

Lin, M., Chen, Q., & Yan, S. (2013). Network in network. arXiv preprint arXiv:1312.4400.

Network in Network (2014)



Network in Network (2014)



GoogLeNet (2014) and InceptionNet

- GoogLeNet (Szegedy et al., 2015), or InceptionNet-V1, was the winning entry of the ILSVRC-2014 classification; 6.7% top-5 error
- with 22 layers GoogLeNet is even deeper than VGG-19
- 6.9 million parameters
 - 5% of the total parameter count in VGG-19
 - Reduction by global average pooling
- Inception-Modules (see later)

Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (2015). Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1-9).

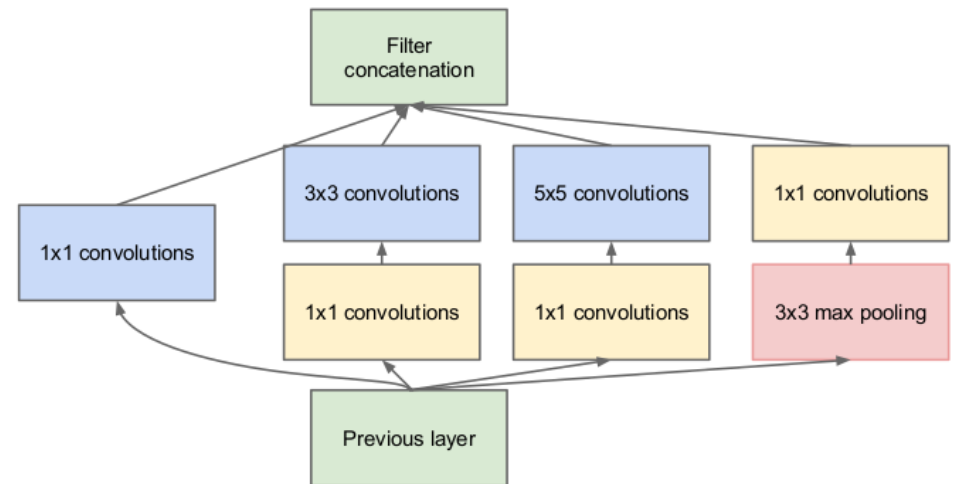
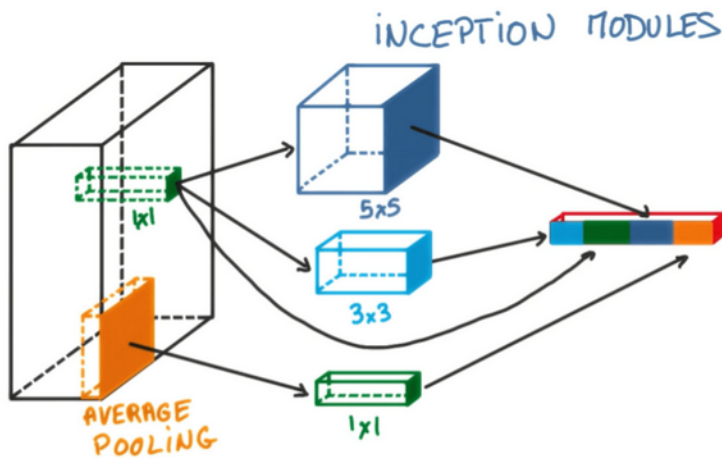
GoogLeNet (2014) and InceptionNet

- Comparison of # weights to VGG

	VGG	M weights	GoogLeNet	M weights
			Global Average Pooling	0
FC-1	$(7 \times 7 \times 512) \times 4096$	102	1024×1000	1
FC-2	4096×4096	16		
FC-3	4096×1000	4		
total		122		1

GoogLeNet (2014) and InceptionNet

- Inception modules

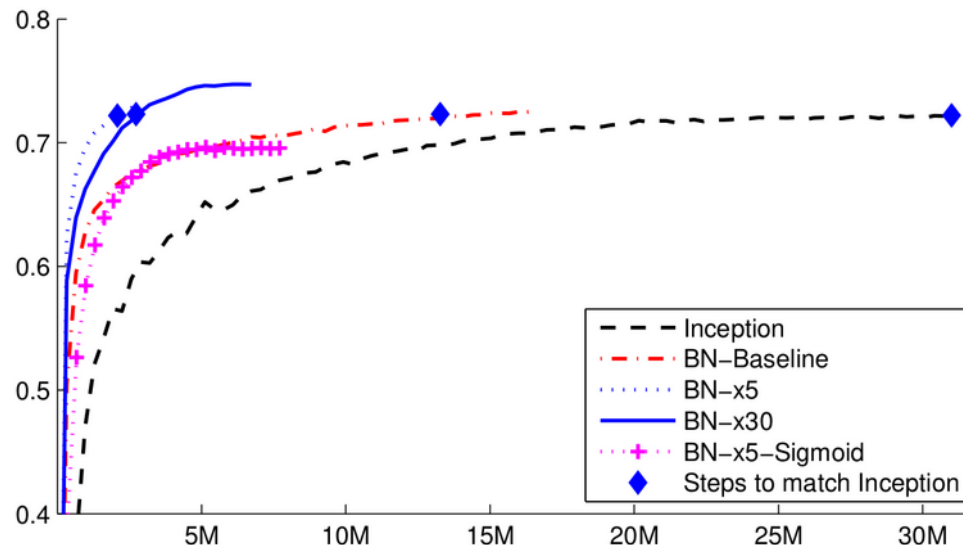


GoogLeNet (2014) and InceptionNet

type	patch size/stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

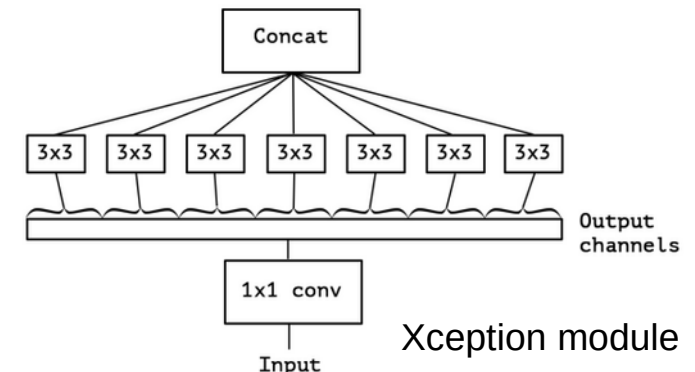
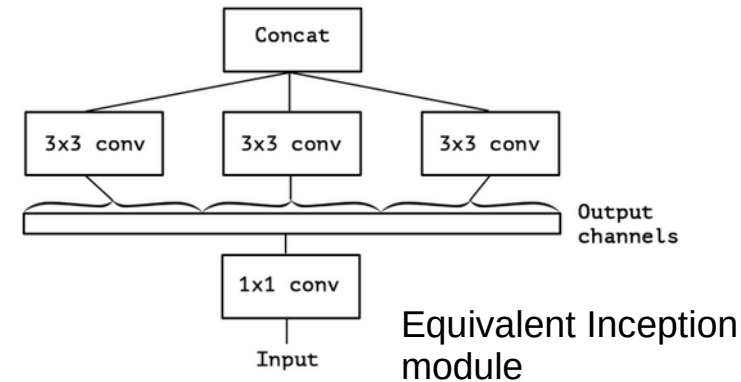
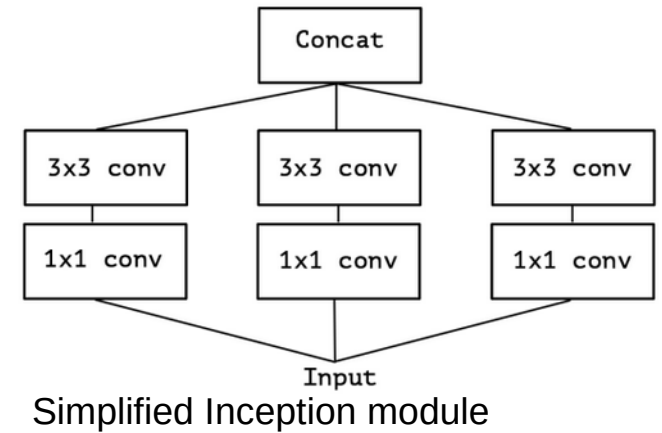
BatchNorm InceptionNet, InceptionNet-V2 (2015)

- InceptionNet combined with Batch Normalization
- Dropout removed
 - Regularizing effect by Batch Normalization sufficient
- Trained 14 times faster than without BN



Xception (2016)

- Introduces *depthwise separable convolutions (DSC)*
- Standard conv:
 - Simultaneous: cross-channel correlations & spatial correlations
- Inception module
 - First: Cross-channel correlations
 - Then: Spatial correlation
- DSC:
 - Pointwise convolution (1x1) mixing channels
 - Spatial conv over each channel



Highway Networks (2015)

- Highway-networks (Srivastava et al., 2015b) are fully-connected or convolutional neural networks
- inspired by the LSTM architecture and employ gates
- Highway archs can be used to train networks up to 100 layers
- “Unimpeded information flow” across many layers is the name giver for these NNs
- Idea: use adaptive gating units to regulate the information flow

Srivastava, R. K., Greff, K., & Schmidhuber, J. (2015). Highway networks. arXiv preprint arXiv:1505.00387.

Highway Networks (2015)

- For feed-forward neural networks, we have a layer:

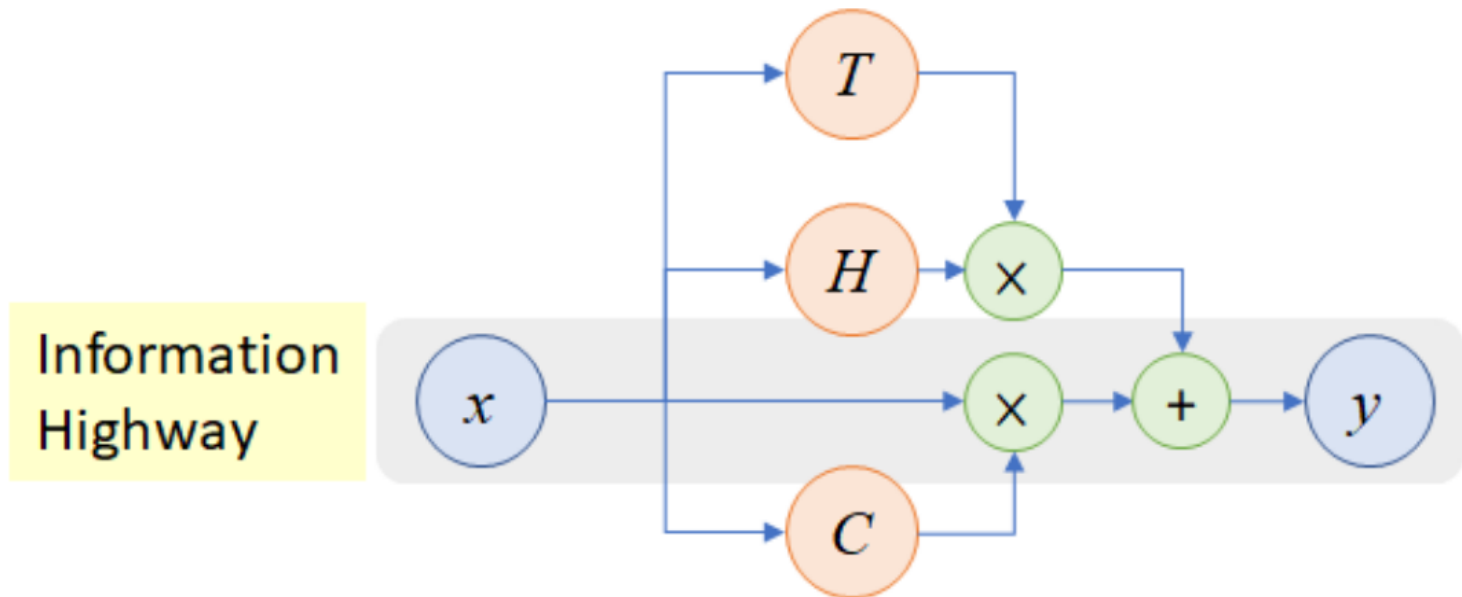
$$\mathbf{a} = H(\mathbf{x}; \mathbf{W}_H) = \mathbf{W}_H \mathbf{X}$$

- Highway Networks use the following:

$$\mathbf{a} = H(\mathbf{x}; \mathbf{W}_H) \odot T(\mathbf{x}, \mathbf{W}_T) + \mathbf{x} \odot (1 - T(\mathbf{x}, \mathbf{W}_T))$$

- Where $H(\mathbf{x}; \mathbf{W}_H)$ is a learned transform, e.g. a convolution or fully-connected layer
- And $T(\mathbf{x}, \mathbf{W}_T)$ is a transform gate with sigmoid outputs
 - Decides whether input is copied or transformed

Highway Networks (2015)



Residual Networks (2015)

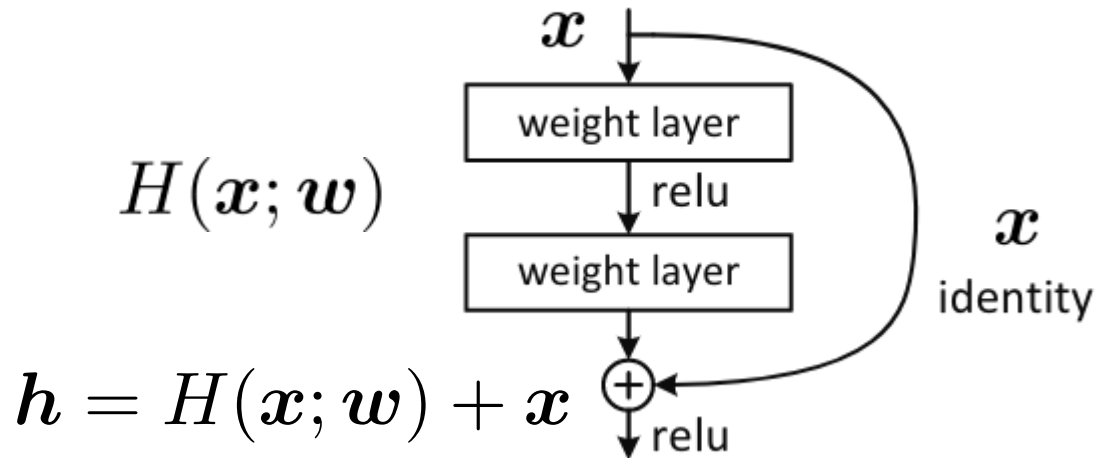
- Residual Networks (He et al., 2015) are a new type of CNNs that use *skip connections*
- The main idea is to model a residual $F(x) - x$ rather than the function $F(x)$ itself.
 - ResNets try to learn a function $H(x; w) = F(x) - x$ or:

$$h = H(x; w) + x$$

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

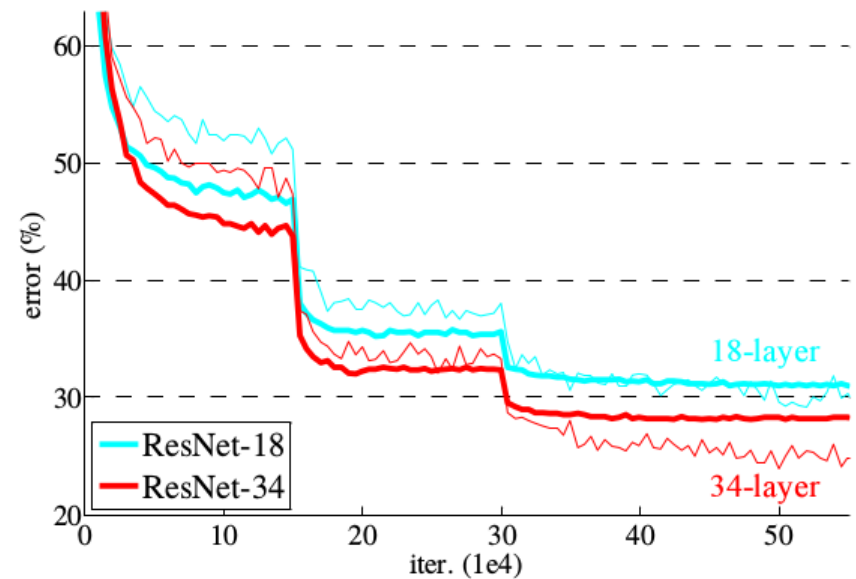
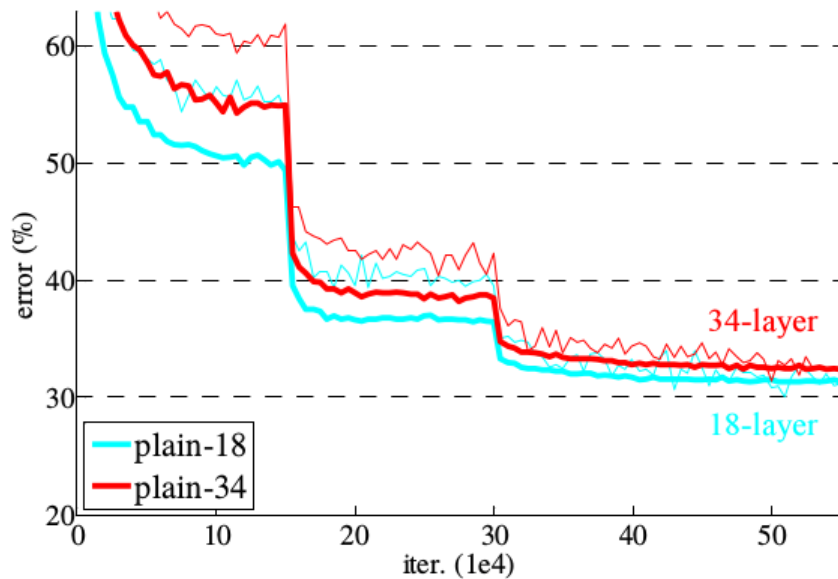
Residual Networks (2015)

- Main operation in ResNets



- The figure illustrates why the addition of the input to a learned transformation is called *skip connection*

Residual Networks (2015): Improvement with many layers

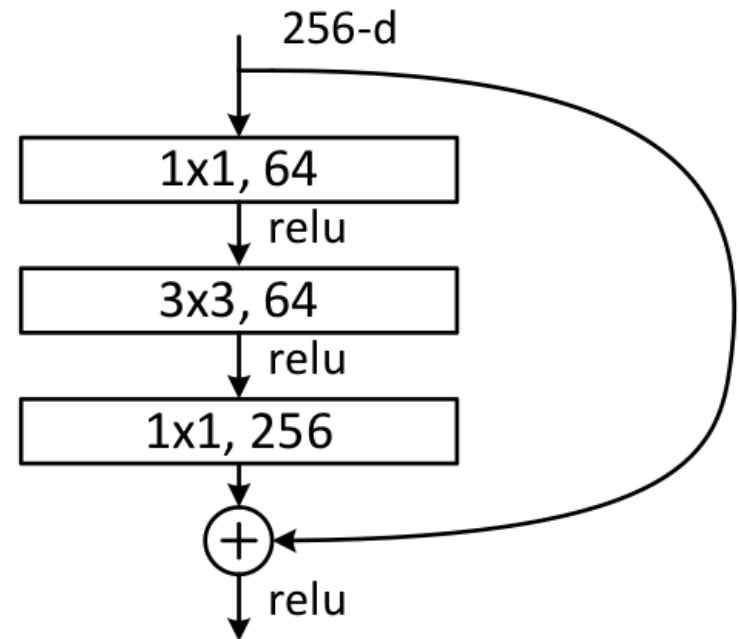
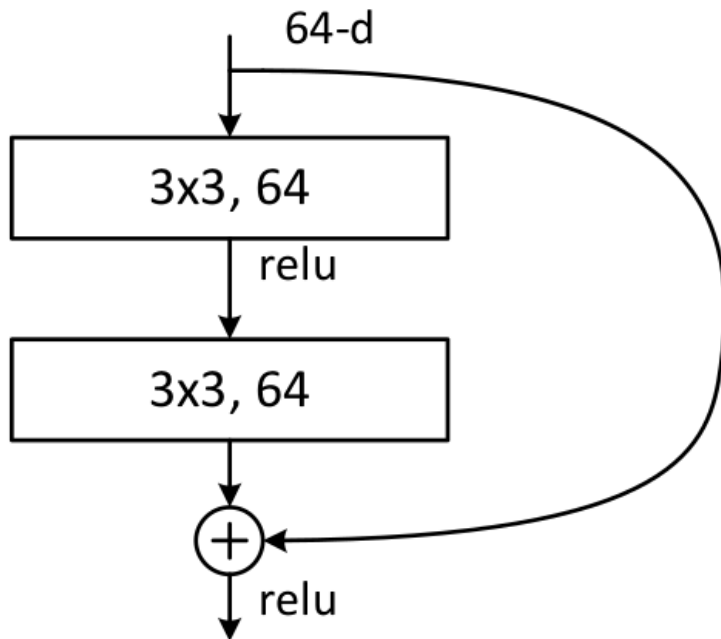


- Performance of ResNets increases even if number of layers is much higher than 20.

Residual Networks (2015): ILSVRC-2015

- First places in all major classification, localization and detection tasks in 2015.
- achieves a human-level error rate of 3.567% (top-5 error)
 - beating out the the runner-up InceptionNet V3 (3.581%)
- especially good at **localization and object detection tasks** (used as backbone of R-CNN)
 - ResNet 9%, 2nd place 12% and
 - won 194 out of 200 categories in the object
- ResNet as backbones won the 2015 segmentation and object detection challenges of the MSCOCO benchmark.
- details on localization and object detection next time

ResNet blocks

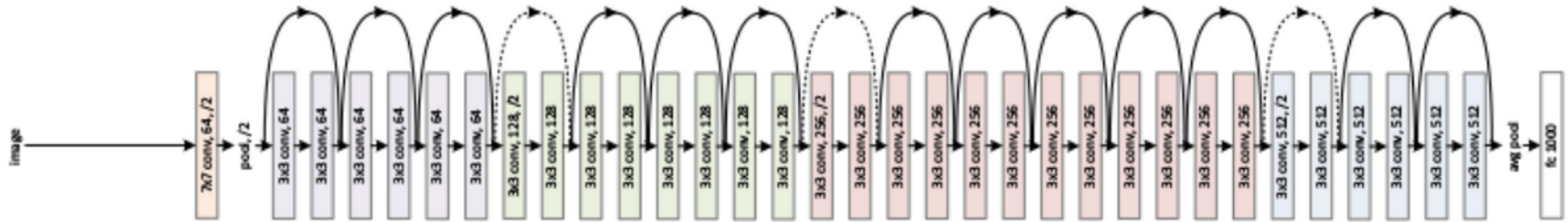


ResNet architectures

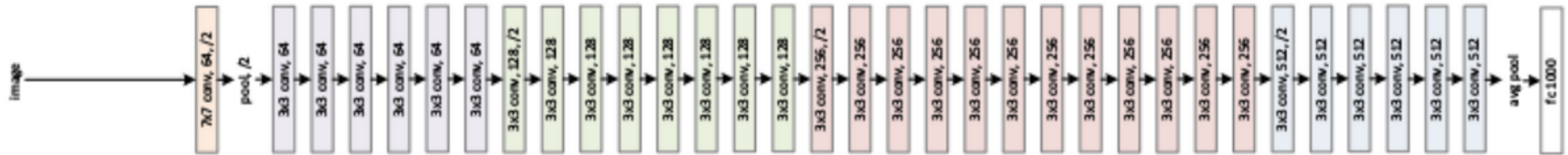
layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	$7 \times 7, 64, \text{stride } 2$				
4*conv2_x	$4 \times 56 \times 56$	$3 \times 3 \text{ max pool, stride } 2$				
		$3 * \begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$3 * \begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$3 * \begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$3 * \begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$3 * \begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
3*conv3_x	$3 \times 28 \times 28$	$3 * \begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$3 * \begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$3 * \begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$3 * \begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$3 * \begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
3*conv4_x	$3 \times 14 \times 14$	$3 * \begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$3 * \begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$3 * \begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$3 * \begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$3 * \begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
3*conv5_x	$3 \times 7 \times 7$	$3 * \begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$3 * \begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$3 * \begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$3 * \begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$3 * \begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ResNet-34 configuration

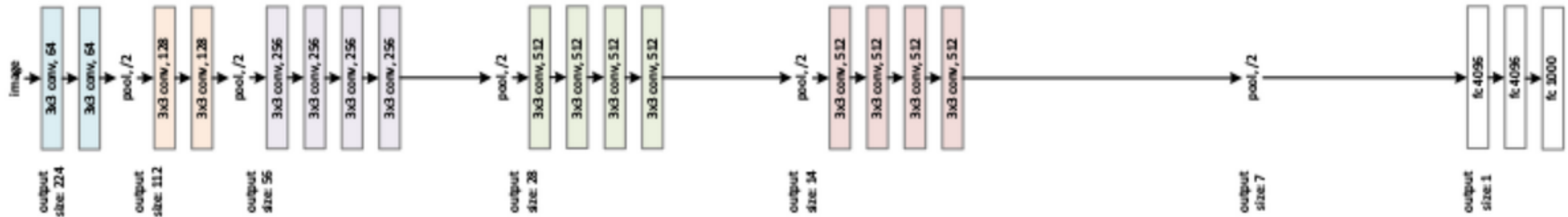
34-layer residual



34-layer plain



VGG-19



ResNet backbones for transfer learning

- Even up-to-now ResNet backbones are frequently used for transfer learning
 - Many Kaggle challenges won with ResNet backbones
 - Adaption easier than for Inception

Pre-activation ResNet (2016)

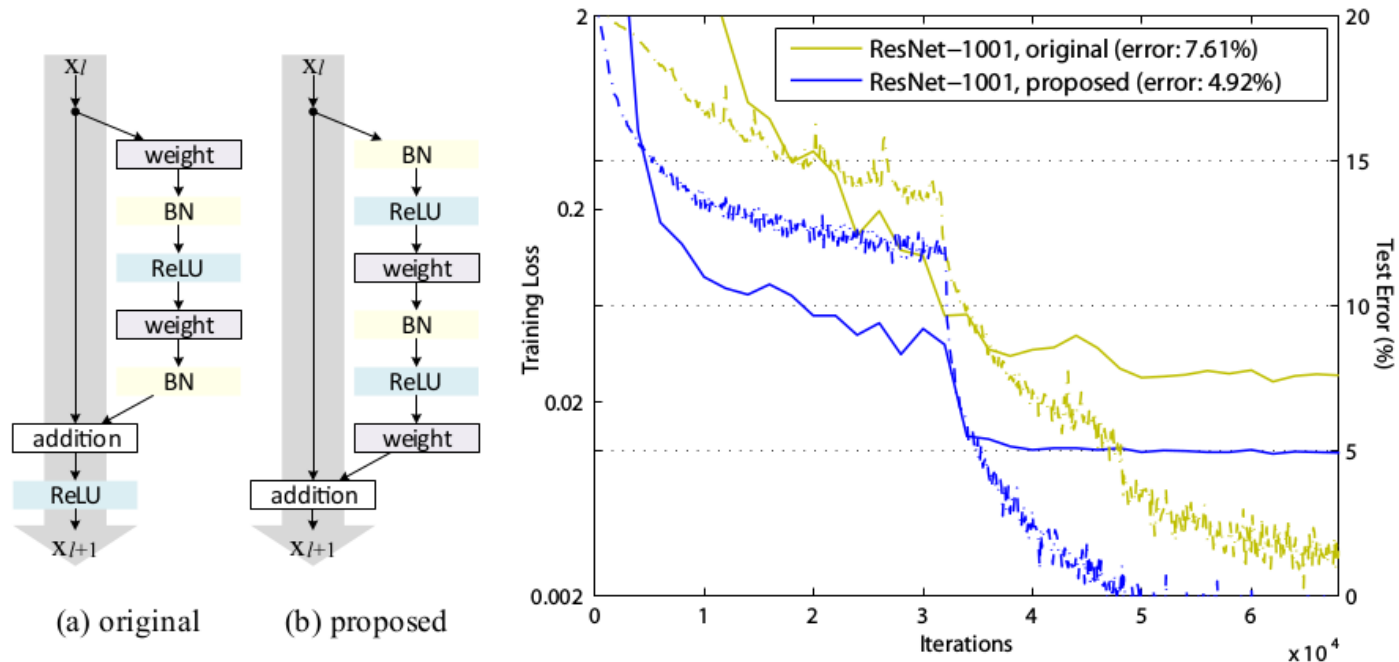


Figure 1.21: original (a) versus improved (b) architecture. 1001 layer results on CIFAR10

- Experiments with ultra-deep networks (1001 layers)
- Changes in residual blocks (see left Figure)

ResNeXt (2016)

- ResNeXt (Xie et al., 2017) took second place in 2016 with a top-5 error of 3.0%.
 - Sidenote: Winner 2016: 2.9% top-5 accuracy
 - Not a novel model, but on the careful selection and calibration of an ensemble consisting of Inception-v3, Inception-v4, Inception-ResNet-v2, Pre-Activation ResNet-200, and Wide ResNet (WRN-68–2) models. Furthermore, extensive test time augmentation techniques have shown considerable gains compared to single-crop evaluation.
- **Grouped convolutions** to redistribute computational cost and parameters in the pre-activation ResNet model
 - Grouped convolutions date back to the AlexNet paper, if not earlier. The motivation given by (Krizhevsky et al., 2012a) is for distributing the model over
 - two GPUs as the model initially did not fit on one single GPU.
 - Main idea: sum over different transformation (before: concatenate transformations)
- ResNeXt modules can be described with three different designs (see later)
- A cardinality parameter controls the number of groups

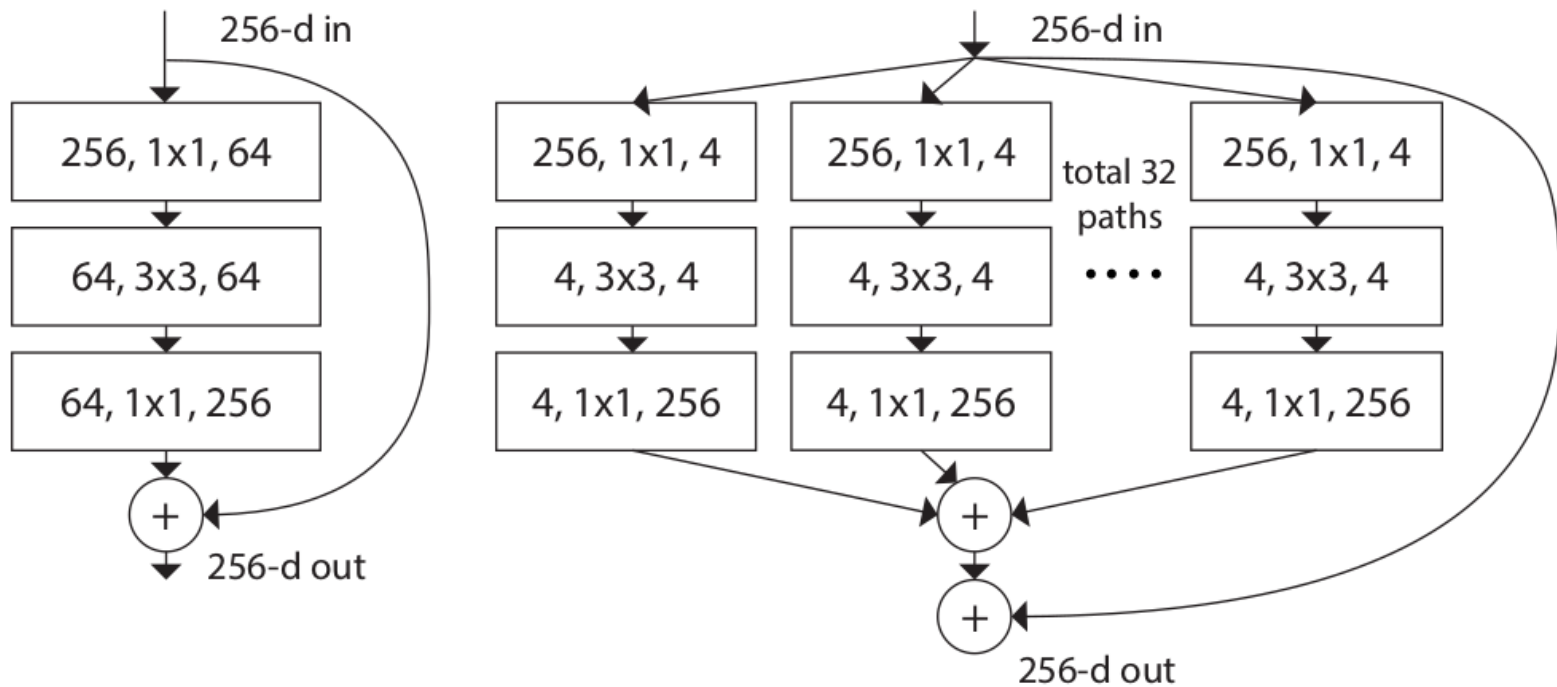
Xie, S., Girshick, R., Dollár, P., Tu, Z., & He, K. (2017). Aggregated residual transformations for deep neural networks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1492-1500).

ResNeXt (2016)

$$H(x; w) = \sum_{c=1}^C T_c(x; w_c)$$

- A transformation from one layer to the next $H(x; w)$
- expressed as sum over individual, different transforms $T_c(x; w_c)$
- “Cardinality parameter”: C

ResNeXT blocks



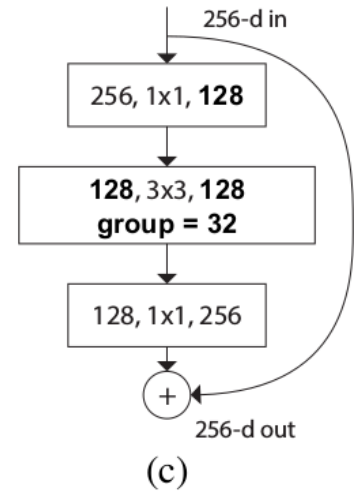
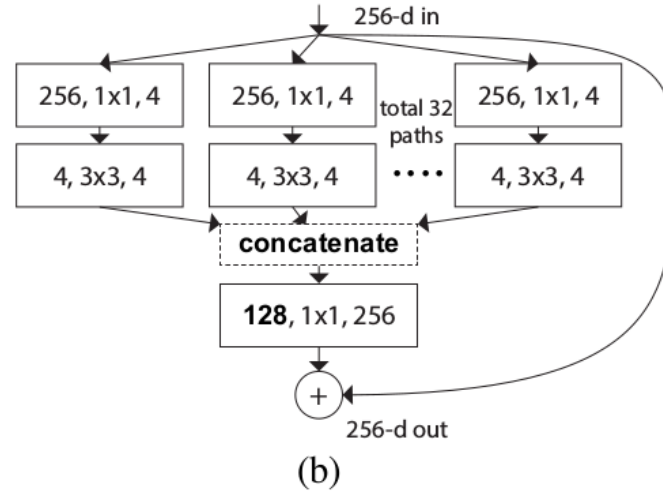
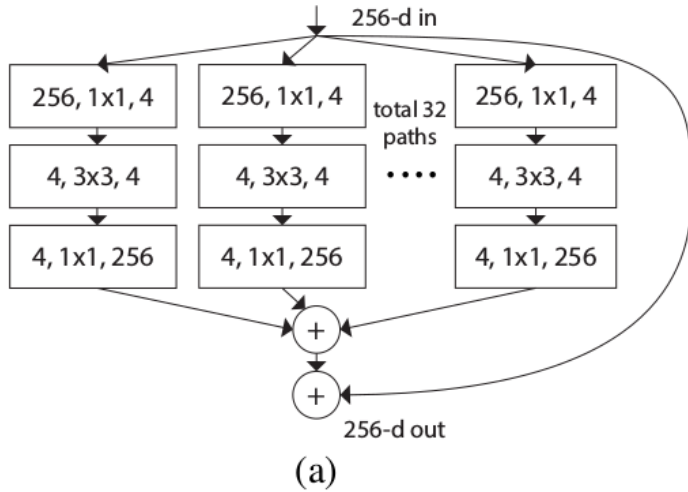
- Left: ResNet block
- Right: ResNext block; summation of transformations

ResNeXt weights

ResNet	weights	ResNeXt (C=32)	× weights
$256 \times 1 \times 1 \times 64$	16,384	$256 \times 1 \times 1 \times 128$	32,768
$64 \times 3 \times 3 \times 64$	36,864	$4 \times 3 \times 3 \times 4 (\times 32)$	4,608
$64 \times 1 \times 1 \times 256$	16,384	$128 \times 1 \times 1 \times 256$	32,768
ResNet	69,632	ResNeXt	70,144

ResNeXt variants

equivalent



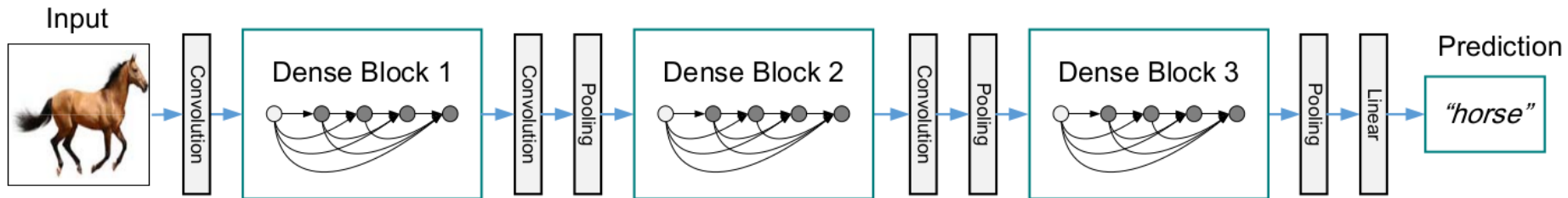
DenseNet (2016)

- DenseNet (Huang et al., 2017) is motivated by the Cascade Correlation algorithm (see DLNN1).
- Main idea is to re-use feature maps from lower layers
 - benefits memory efficiency and provides a direct gradient to feature maps
 - Can learn to combine features of different hierarchies
- classification and in experiments this showed to produce more diversified features and to reduce over-fitting.

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4700-4708).

DenseNet (2016)

- Dense blocks in DenseNet



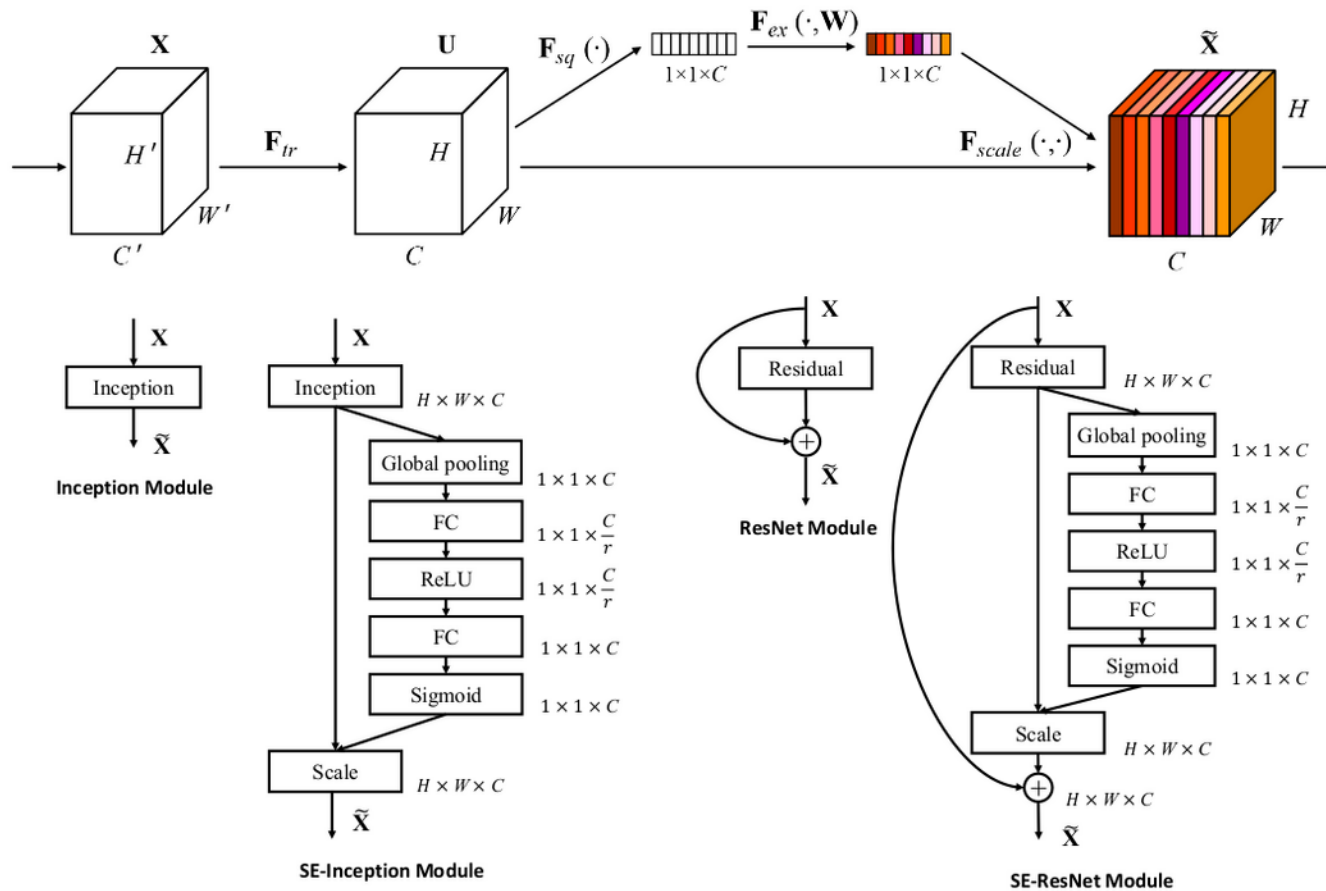
- Main operation
 - ResNets: $x^{[l+1]} = H(x^{[l]}; w) + x^{[l]}$
 - DenseNet: $x^{[l+1]} = H([x^{[0]}, \dots, x^{[l]}]; w)$
where the lower level representations are concatenated

Squeeze and Excitation Net (2017)

- SENet (Hu et al., 2018) is the winner of ILSVRC-2017 with a top-5 error of 2.2%.
- Squeeze: Performs global average pooling in every convolutional block
- Excite: learns to re-scale the feature maps based on the per-channel sigmoid activations

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 7132-7141).

Squeeze and Excitation Net (2017)

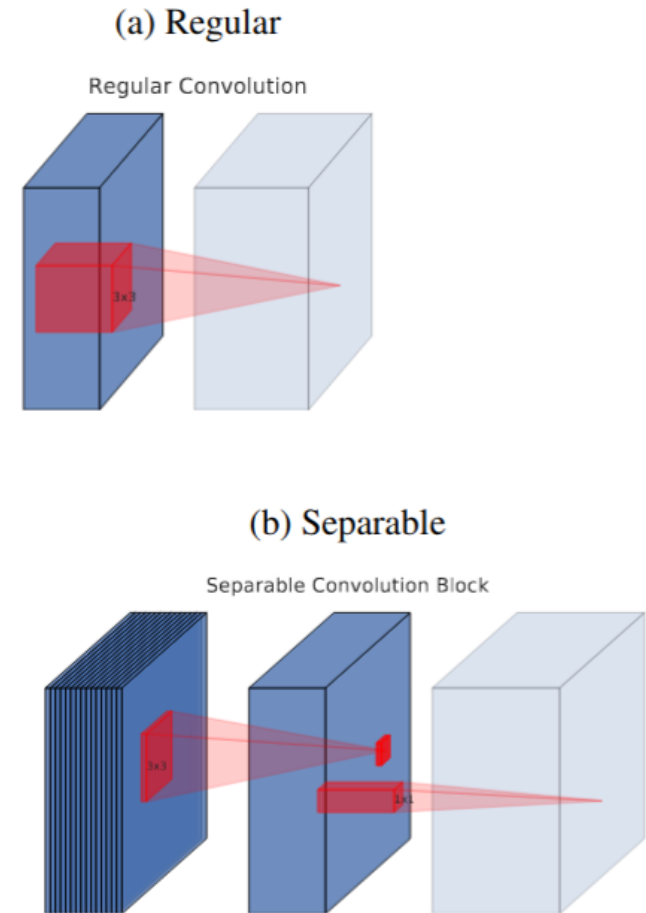


ILSVRC comes to an end

- Yearly ILSVRC comes to an end. As of 2017, the yearly ILSVRC challenge came to an end.
- Taking annotation errors into account ($\sim 1\%$ bad labels), top-5 classification was close to perfect
- and reached a level far above human performance (Figure 1.2). Research focus shifted either
- towards top-1 error, or towards developing more efficient models that enable to trade a small drop
- in accuracy for a large gain in inference speed.

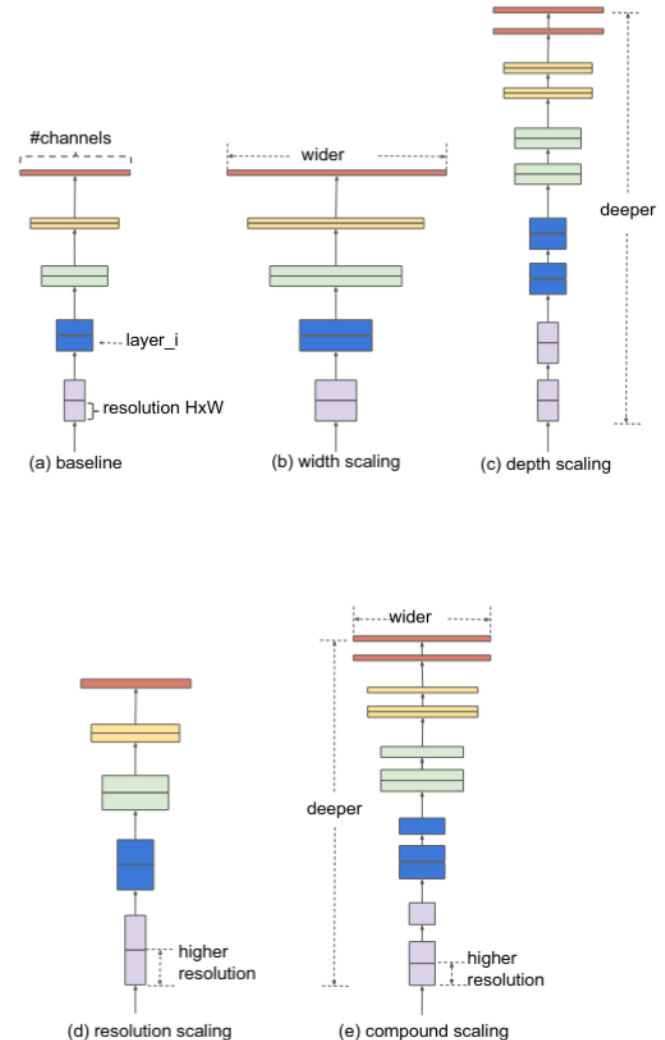
MobileNetV1 and MobileNetV2 (2018)

- Focus on minimize compute and inference time
- Depthwise separable convolutions
 - Go back to XceptionNet
- Skip-connections and bottlenecks
- 74.7% top-1 accuracy in ImageNet
 - 143ms per image on CPU



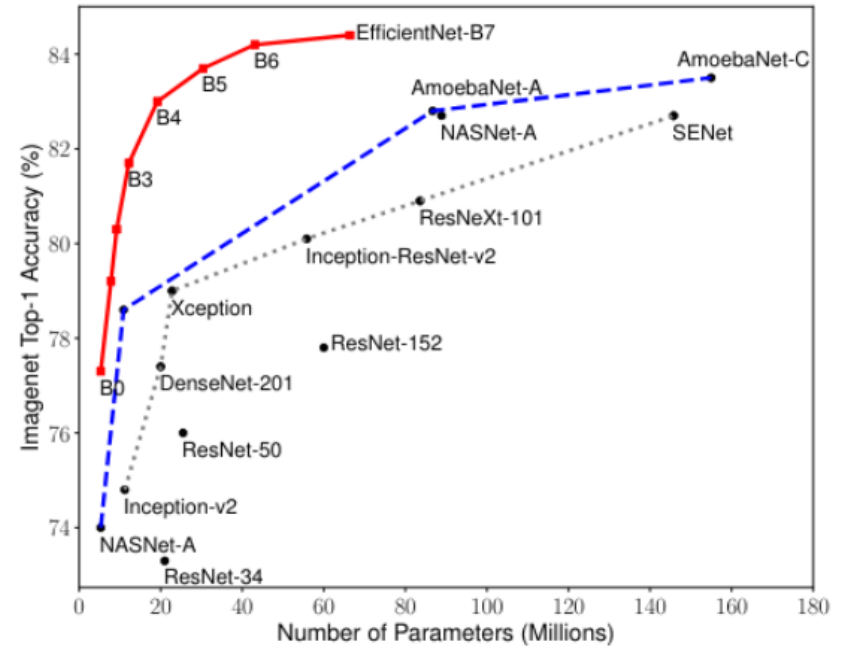
EfficientNet

- Neural architecture search (NAS) used to find an efficient architecture
- Huge hyperparameter search space, e.g.
 - Depth (number of layers)
 - Width (number of channels)
 - Resolution (size of input image)
- Combined in one “compound scaling” hyperparameter
- Uses inverse residual blocks from MobileNetV2 and “SE modules” from SENet



EfficientNet (2019)

- Neural architecture search (NAS) used to find an efficient architecture
- Huge hyperparameter search space, e.g.
 - Depth (number of layers)
 - Width (number of channels)
 - Resolution (size of input image)
- Combined in one “compound scaling” hyperparameter
- Uses inverse residual blocks from MobileNetV2 and “SE modules” from SENet



Transformer architectures for vision

- *Input set* instead of single inputs: $\mathbf{X} = \{\mathbf{x}(1), \dots, \mathbf{x}(T)\}$

$$\mathbf{K}^{(l)} = \mathbf{W}_K^{(l)} \mathbf{X}^{(l-1)} \quad (11.6)$$

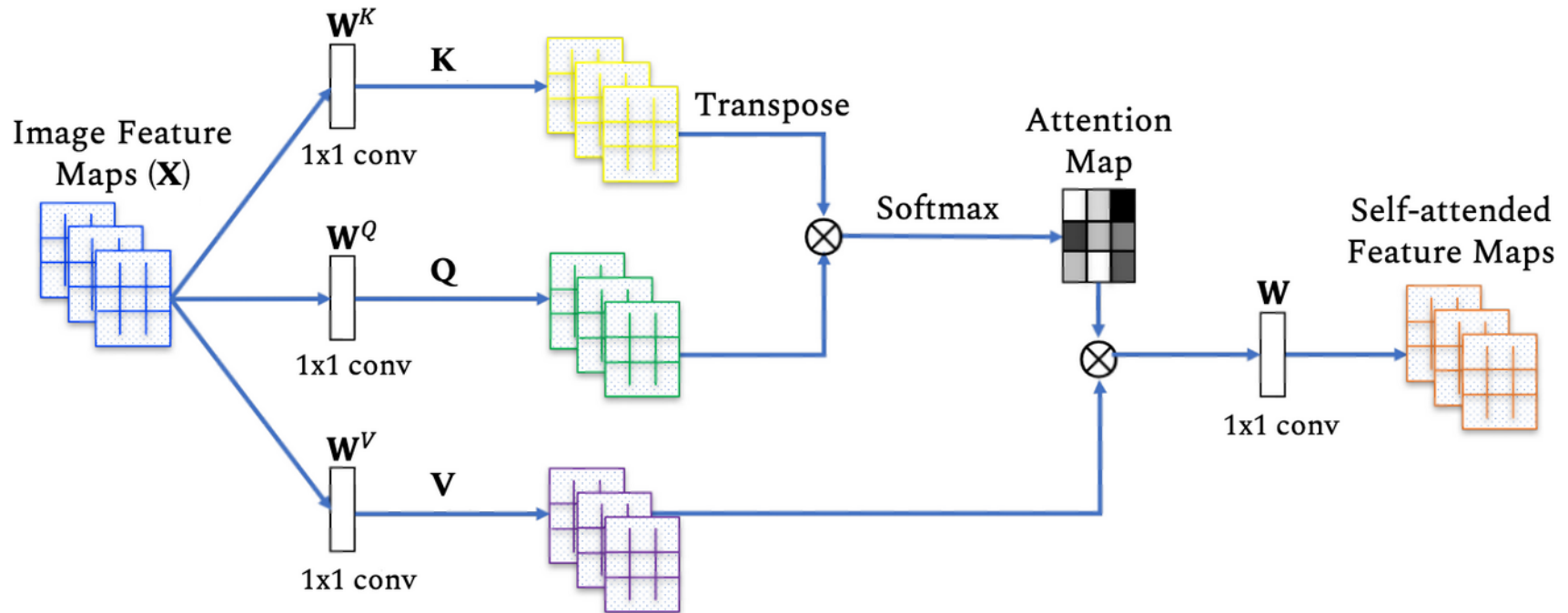
$$\mathbf{Q}^{(l)} = \mathbf{W}_Q^{(l)} \mathbf{X}^{(l-1)} \quad (11.7)$$

$$\mathbf{V}^{(l)} = \mathbf{W}_V^{(l)} \mathbf{X}^{(l-1)} \quad (11.8)$$

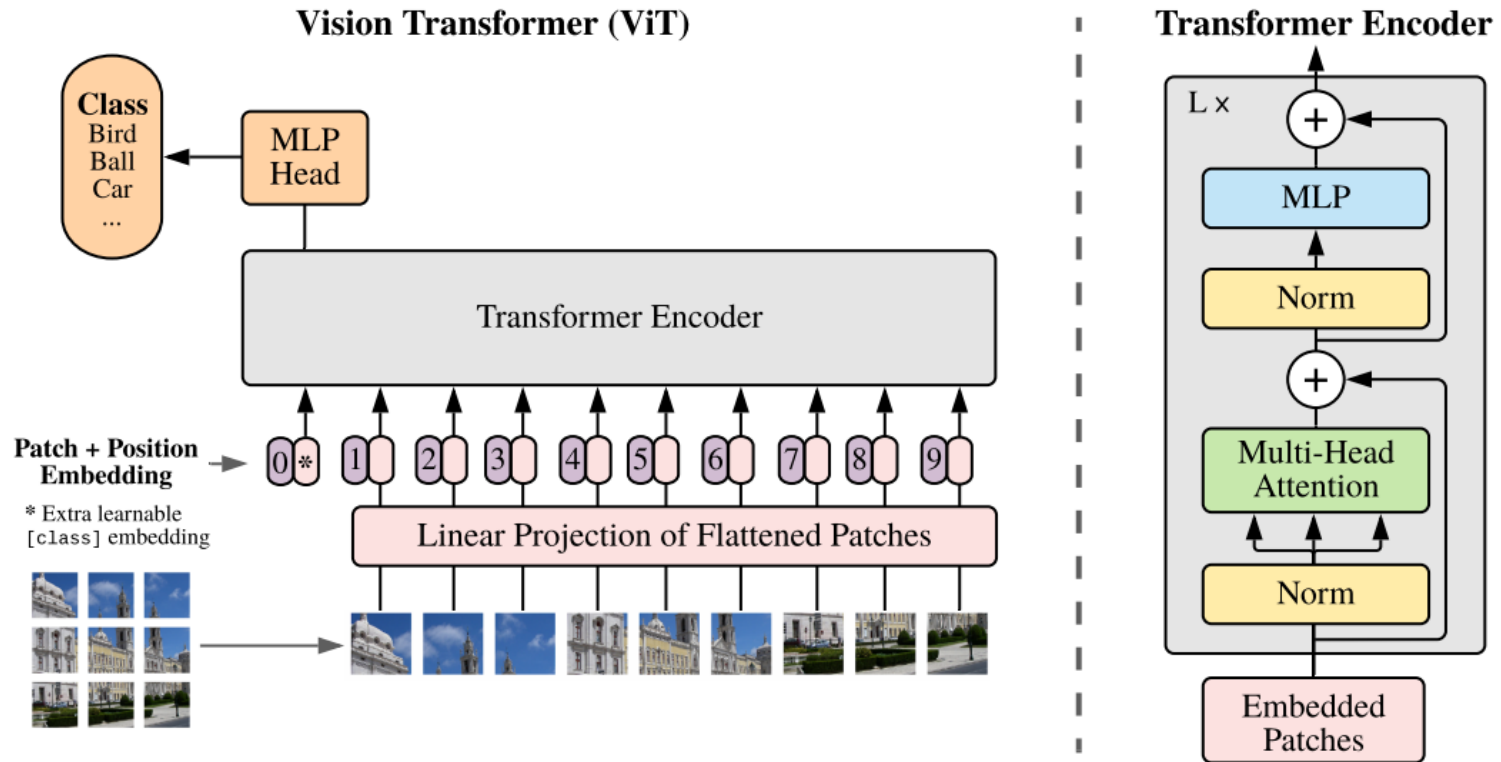
$$\mathbf{X}^{(l)} = \underbrace{\text{softmax} \left(1/\sqrt{d_k} \mathbf{Q}^{(l)T} \mathbf{K}^{(l)} \right)}_{:=\mathbf{A}^{(l)}} \mathbf{V}^{(l)}, \quad (11.9)$$

where $\mathbf{X}^{(l-1)} \in \mathbb{R}^{d \times T}$ is the input set, $\mathbf{W}_K^{(l)} \in \mathbb{R}^{d_k \times d}$ is the *keys embedding matrix*, $\mathbf{W}_Q^{(l)} \in \mathbb{R}^{d_k \times d}$ is the *query embedding matrix*, $\mathbf{W}_V^{(l)} \in \mathbb{R}^{d_v \times d}$ is the *value embedding matrix*, $\mathbf{Q}^{(l)} \in \mathbb{R}^{d_k \times T}$ are the *keys*, $\mathbf{K}^{(l)} \in \mathbb{R}^{d_k \times T}$ are the *queries*, $\mathbf{V}^{(l)} \in \mathbb{R}^{d_v \times T}$ are the *values*, $\mathbf{A}^{(l)} \in \mathbb{R}^{T \times T}$ is the *attention matrix*, and $\mathbf{X}^{(l)} \in \mathbb{R}^{d_v \times T}$ is the representation of the input set in the next layer.

Transformer architectures for vision

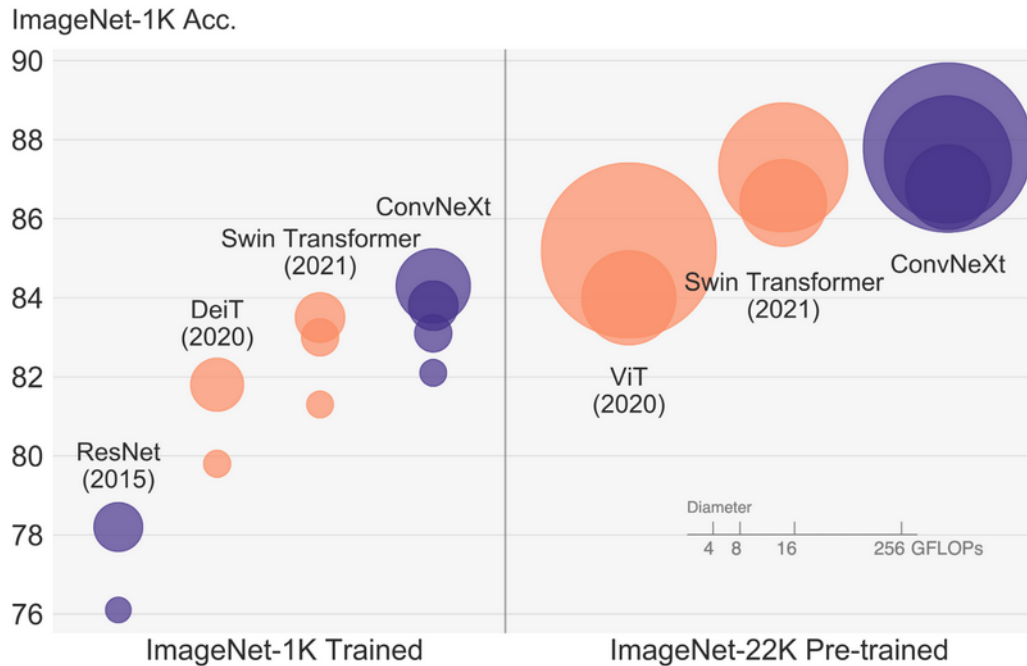


Vision Transformer (ViT, 2021)

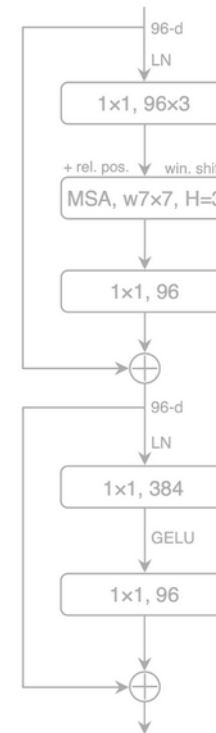


- Complete absence of convolution operations

ConvNext (2022)



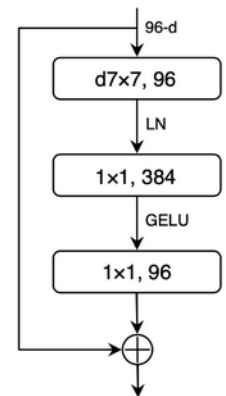
Swin Transformer Block



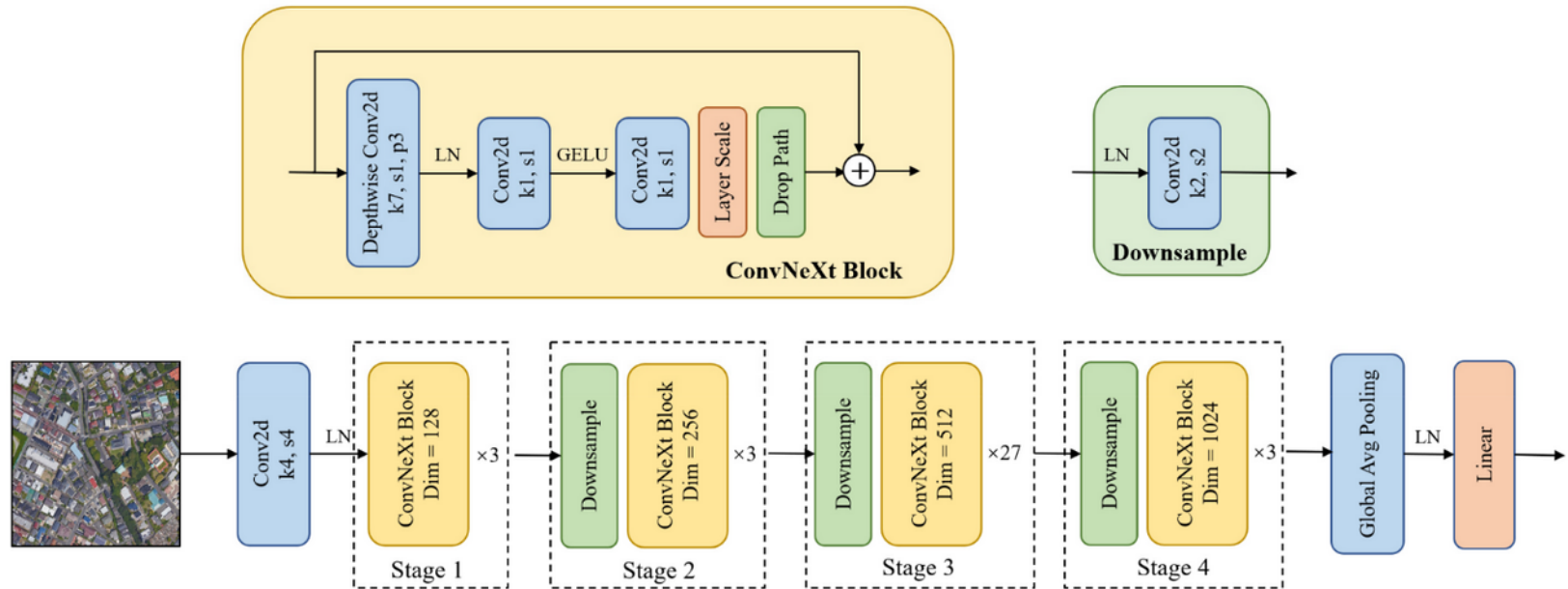
ResNet Block



ConvNeXt Block



ConvNext (2022)



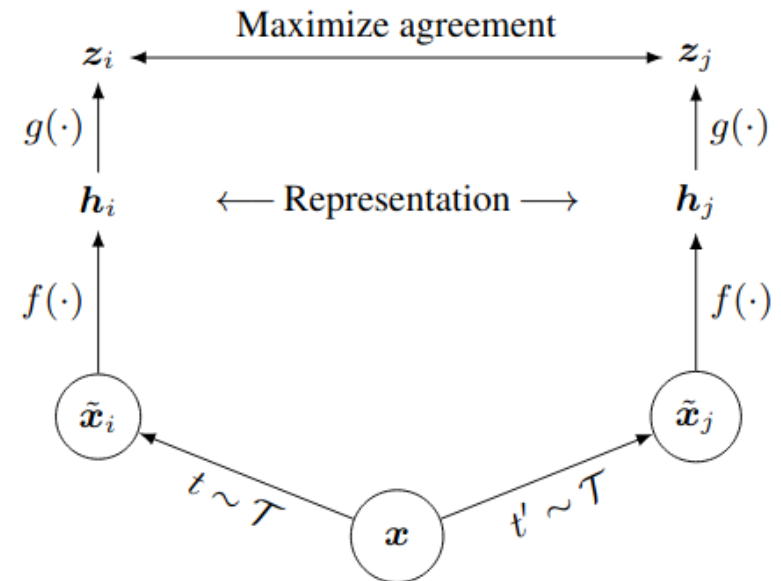
- Training technique: AdamW optimizer
- Macro design (“stages”)
- Inverted bottlenecks
- Large kernel (size 7)
- ReLU → GELU
- Fewer activations
- BatchNorm → LayerNorm
- Depthwise separable convs (Xception)

New learning paradigms

- From around 2020, training deep neural networks on ImageNet has been replaced by other approaches
- Self-supervised “pre-training”
 - Train a network on a general task before
 - Fine-tune to particular task at hand (“downstream task”)
- Pre-training tasks; two main paradigms
 - Contrastive learning or instance-discrimination
 - Masked image modeling

Contrastive learning and contrastive loss

- Labels for image datasets a relatively costly: a human has to look over all images and annotate them
 - still: only few bits of information per image
- **Discriminative approach:**
 - learn representations using objective functions similar to those used for supervised learning
 - but perform tasks where both the inputs and labels are derived from an unlabeled dataset
 - Contrastive learning

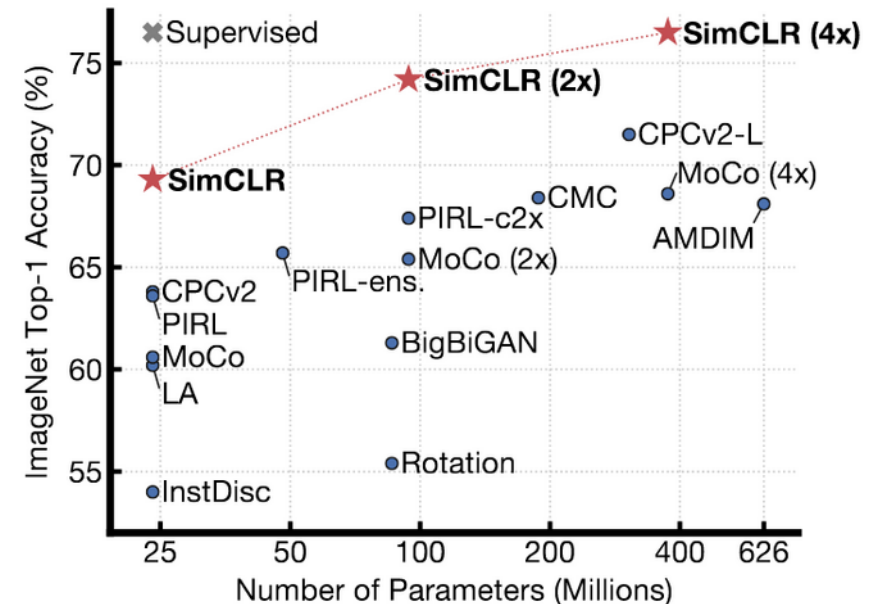


Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020, November). A simple framework for contrastive learning of visual representations. In International conference on machine learning (pp. 1597-1607). PMLR.

SimCLR: An example of a contrastive self-supervised learning algorithm

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N , constant τ , structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{x_k\}_{k=1}^N$ **do**
 for all $k \in \{1, \dots, N\}$ **do**
 draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
 # the first augmentation
 $\tilde{x}_{2k-1} = t(x_k)$
 $h_{2k-1} = f(\tilde{x}_{2k-1})$ # representation
 $z_{2k-1} = g(h_{2k-1})$ # projection
 # the second augmentation
 $\tilde{x}_{2k} = t'(x_k)$
 $h_{2k} = f(\tilde{x}_{2k})$ # representation
 $z_{2k} = g(h_{2k})$ # projection
 end for
 for all $i \in \{1, \dots, 2N\}$ and $j \in \{1, \dots, 2N\}$ **do**
 $s_{i,j} = z_i^\top z_j / (\|z_i\| \|z_j\|)$ # pairwise similarity
 end for
 define $\ell(i, j)$ **as** $\ell(i, j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
 $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^N [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
 update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$



Does not follow this lecture's notation!

Cosine similarity and InfoNCE loss

- Cosine similarity

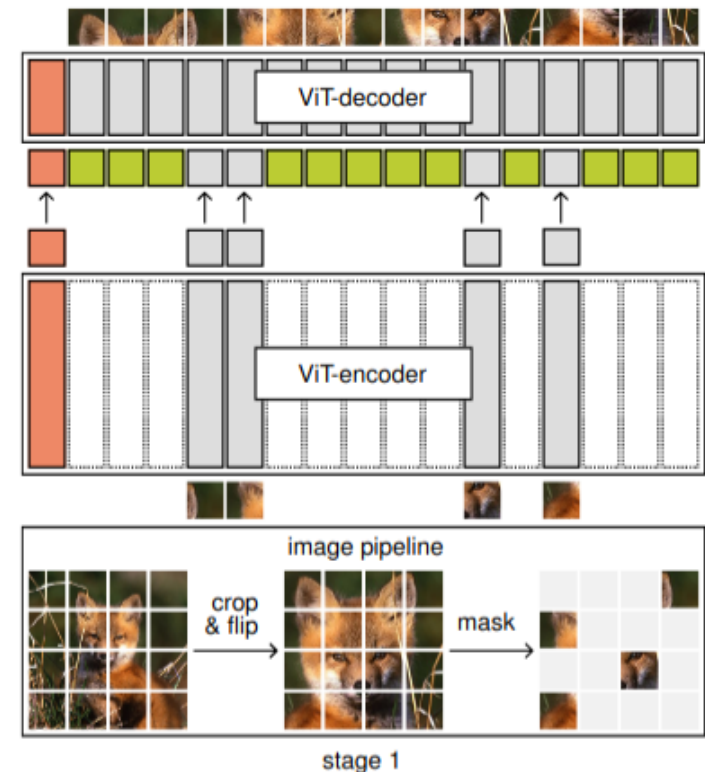
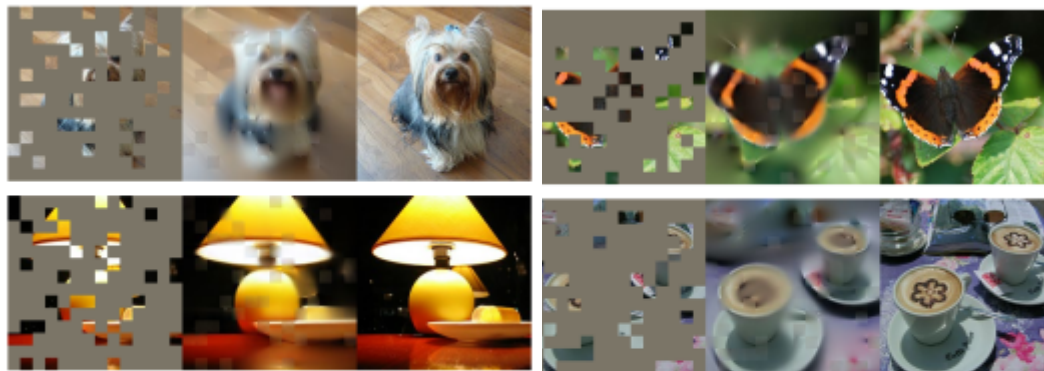
$$\cos(\mathbf{h}^i, \mathbf{h}^j) = \frac{\mathbf{h}^{iT} \mathbf{h}^j}{\|\mathbf{h}^i\| \|\mathbf{h}^j\|}$$

- Contrastive loss, InfoNCE loss:

$$L((\mathbf{h}^0, \mathbf{h}^j), \{\mathbf{h}^l\}_{l=1}^L) = -\log \left(\frac{\exp(1/\tau \cos(\mathbf{h}^0, \mathbf{h}^j))}{\sum_{l=1}^L \exp(1/\tau \cos(\mathbf{h}^0, \mathbf{h}^l))} \right)$$

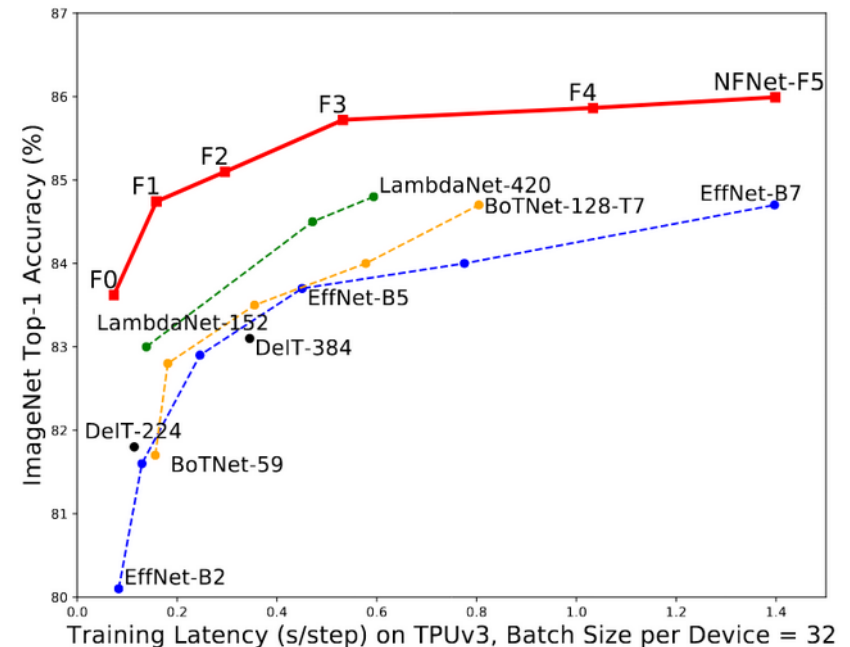
Masked image modeling

- Principle design of a *masked autoencoder*
 - See chapter 12
- Asymmetric encoder-decoder design
- Fits well with the ViT model



Remark: Normalization-free architectures, NFnets

- BatchNorm and other external normalization methods have several drawbacks
- Aim: train a deep, but still accurate network without normalization
- Nfnet is a ResNet
 - 9 times faster to train
 - 86.5% top-1 accuracy
 - modified “SE-ResNeXt-D”
- Method: Adaptive Gradient Clipping (AGC)
 - (without details)



Now: ConvNets match vision transformers at scale

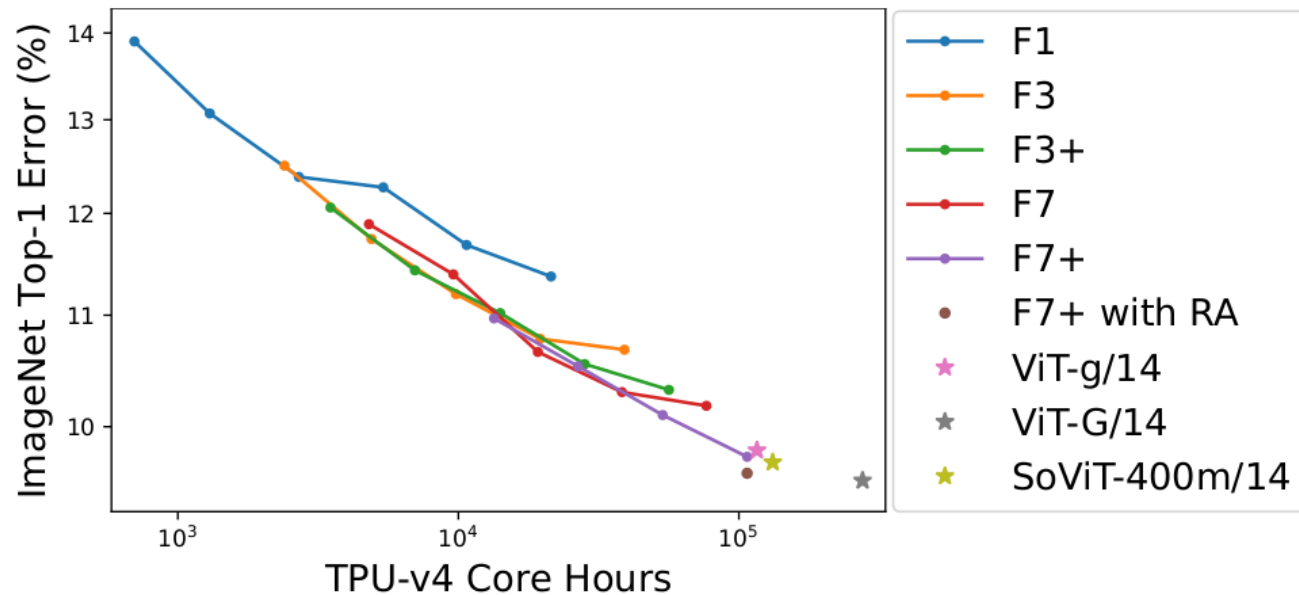


Figure 1 | ImageNet Top-1 error, after fine-tuning pre-trained NFNet models for 50 epochs. Both axes are log-scaled. Performance improves consistently as the compute used during pre-training increases. Our largest model (F7+) achieves comparable performance to that reported for pre-trained ViTs with a similar compute budget (Alabdulmohsin et al., 2023; Zhai et al., 2022). The performance of this model improved further when fine-tuned with repeated augmentation (RA) (Hoffer et al., 2019).

“Scaling Laws”

Now: Large-scale comparison of pre-trained models/backbones

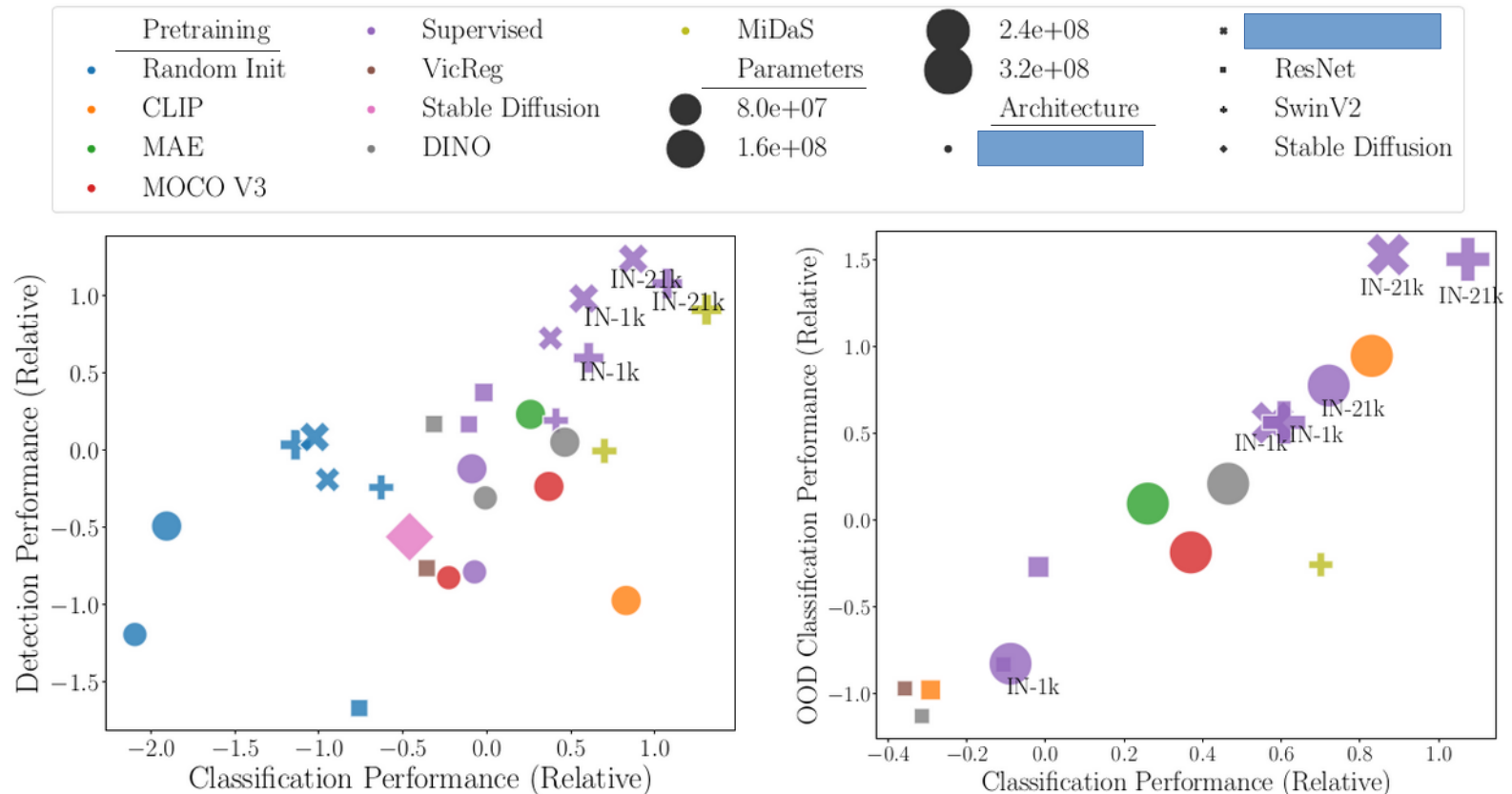


Figure 1: **Performance is correlated across tasks.** Performance for each model is per-dataset standard deviations above the mean averages across datasets. **Left:** Comparison between classification and detection. **Right:** Comparison between classification and OOD classification.

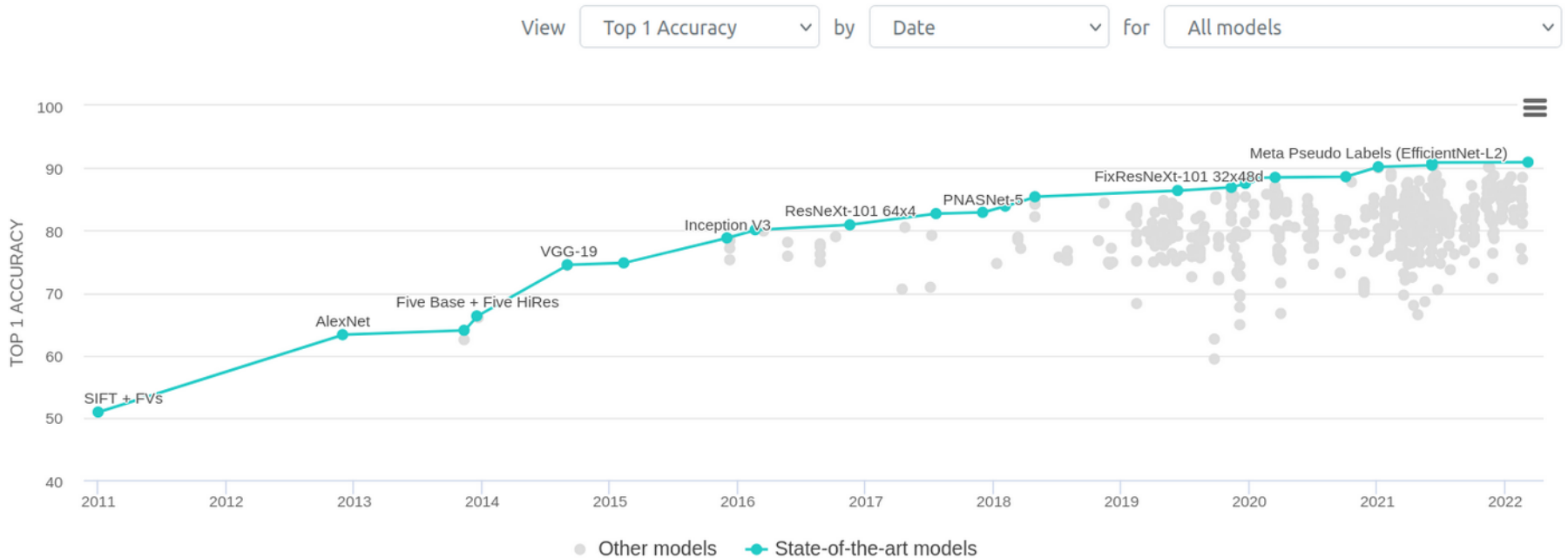
Analysis of 300 ML competitions of 2023

- As expected, almost all winners used Python.
- 92% of deep learning solutions used PyTorch. The remaining 8% we found used TensorFlow,
- CNN-based models won more computer vision competitions than Transformer-based ones.
- **Compute usage varies a lot.** NVIDIA GPUs are obviously common; a couple of winners used TPUs; we didn't find any winners using AMD GPUs;

Current research

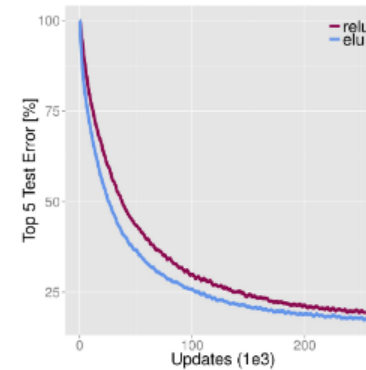
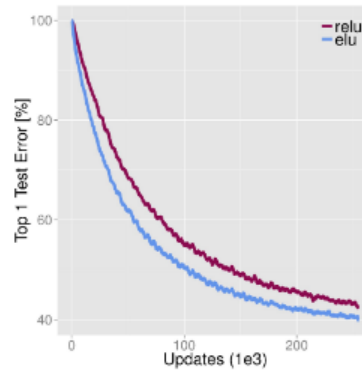
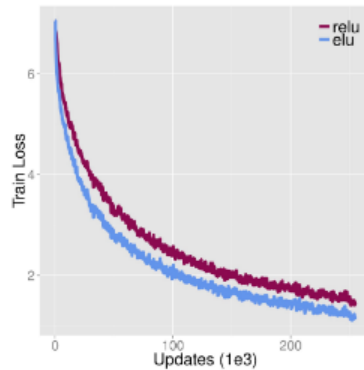
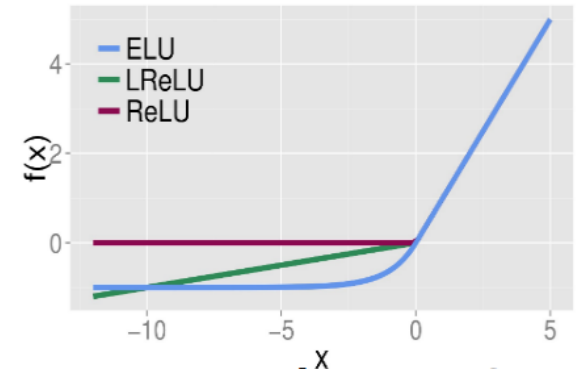
- Most research projects use ResNet50 or ViTs as standard architectures
- Increase efficiency of networks
 - Building small networks (“MobileNet”) with the same performance
 - Fast adaption to new tasks, “few-shot learning”
 - Learn general features for any task “zero-shot transfer learning”
- Faster training
- Faster inference to enable real-time predictions

Current research

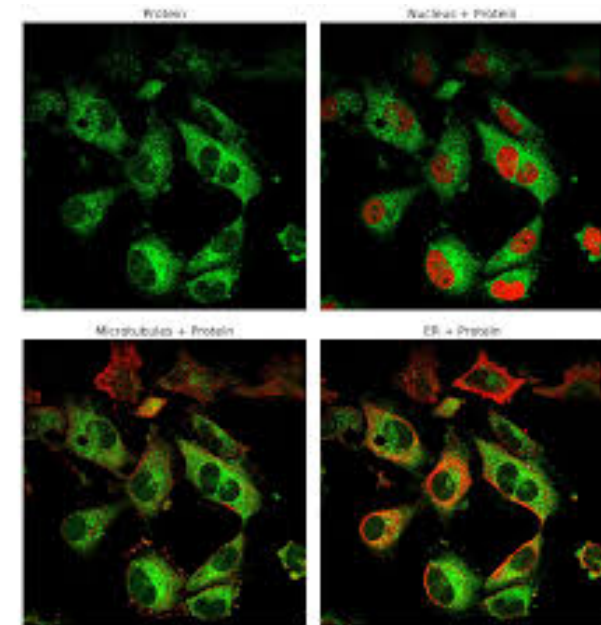
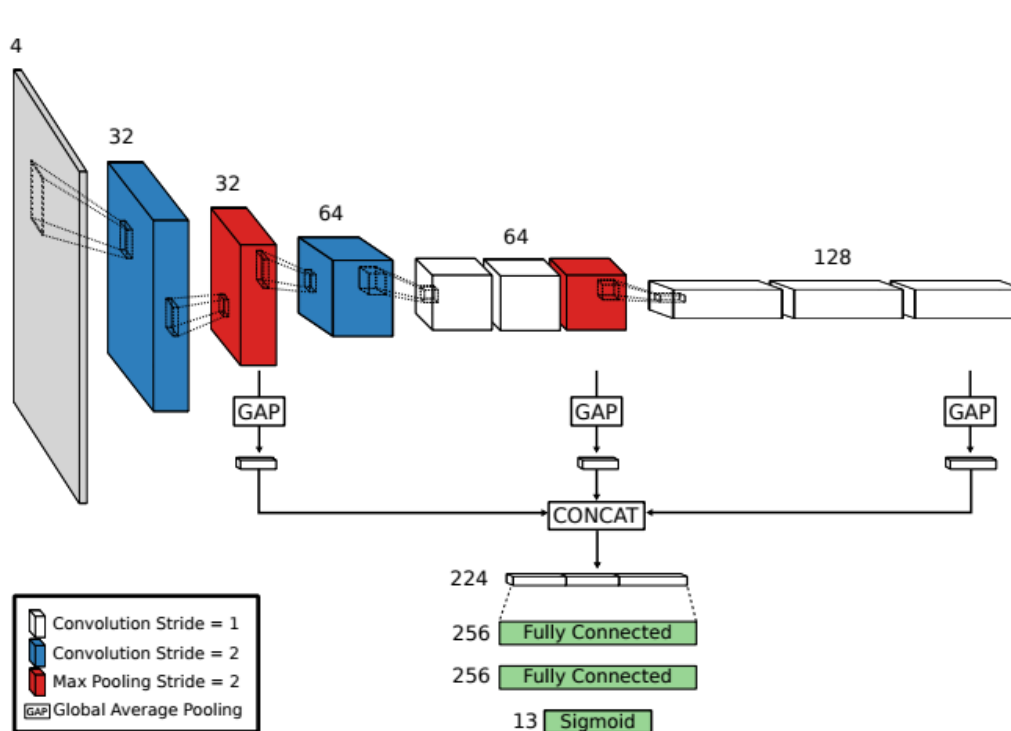


IML and the ILSVRC

- We participated in 2015
 - Introduced exponential-linear units (ELU)
 - 15 layer CNN
 - Best academic group with 9.18% test classification error rate.

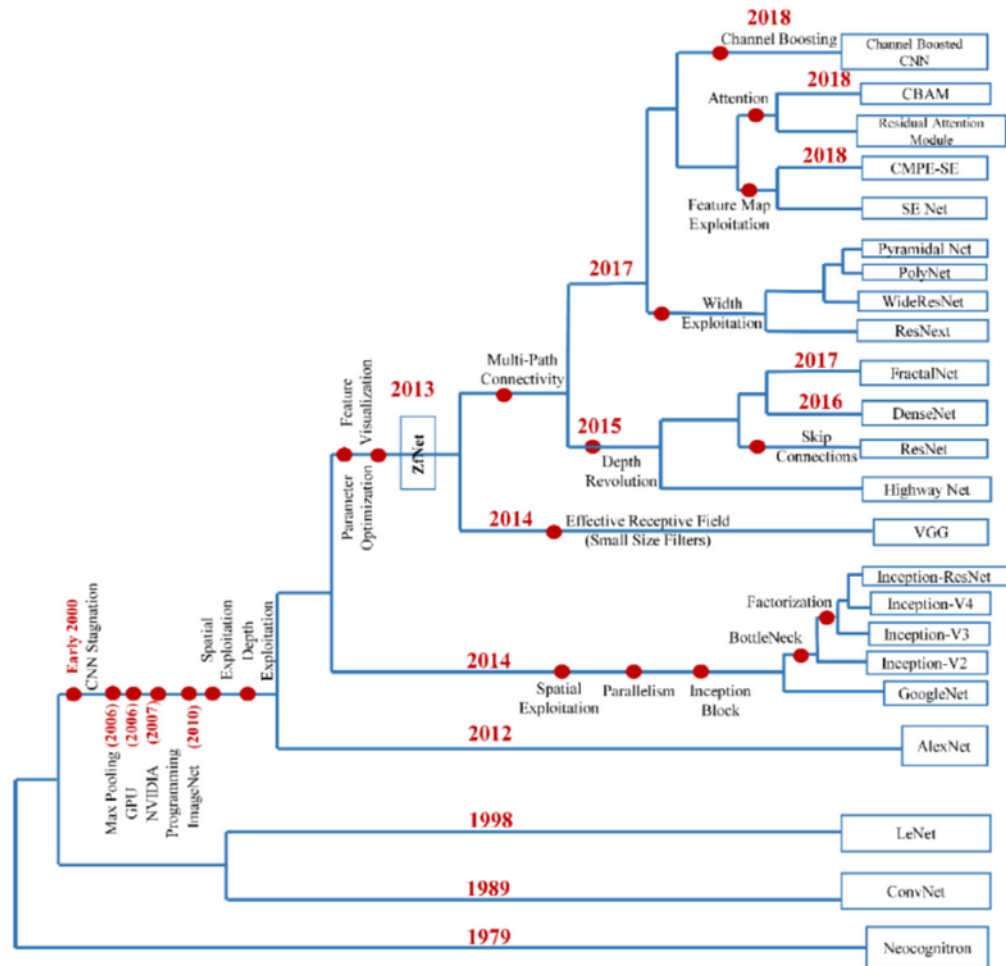


GAPnet



Rumetshofer, E., Hofmarcher, M., Röhl, C., Hochreiter, S., & Klambauer, G. (2018, September). Human-level protein localization with convolutional neural networks. In International Conference on Learning Representations.

Conclusions and Summary



Conclusions and Summary

- **Small kernels.** It is beneficial to replace a convolutional block with large kernel size with multiple 3×3 convolutional blocks. The reasons include that it alleviates aliasing effects, increases representational power, and reduces total number of parameters.
- **Deep networks.** Directly transporting inputs across layers, as in Highway Networks, or in residual blocks or skip-connections have enabled the training of very deep networks.
 - Relation to VGP (see Theory chapter later)
- **Global average pooling and FC layers.** The first FC layer after the convolutions had many parameters.
 - Reduces parameters; decreases overfitting; computationally efficient
- **1x1 convolutions.** These are pixel-wise fully-connected blocks between two layers. They do not change the spatial dimensions, but the number of feature maps.

Conclusions and Summary

- **Bottle-neck 1×1 convolutions.** Reduction of dimensions before more costly $R \times R$ convolutions. Simulate sparse networks on hardware that is optimized towards dense computation.
- **Grouped 3×3 convolutions.**
 - redistribute the compute towards wider layer, increased number of channels, and therefore more costly 1×1 convolutions
 - reduce compute during the initially more costly aggregation of neighbourhood information via sparsely-connected 3×3 convolutions.
- **ResNet breaks with the idea of identity mapping.** At the input of the network and where spatial down-sampling is performed because the dimensions do not match and the addition operation cannot be performed.
- **Recent developments: “2020 – the year of the contrastive loss”**
 - Vision transformers
 - Self-supervised pre-training and contrastive learning